HABS analysis

2026-07-09

```r
library(dplyr)
```

```
##
## Attaching package: 'dplyr'
```

```
## The following objects are masked from 'package:stats':
##
##     filter, lag
```

```
## The following objects are masked from 'package:base':
##
##     intersect, setdiff, setequal, union
```

```r
library(miceadds)
```

```
## Loading required package: mice
```

```
##
## Attaching package: 'mice'
```

```
## The following object is masked from 'package:stats':
##
##     filter
```

```
## The following objects are masked from 'package:base':
##
##     cbind, rbind
```

```
## * miceadds 3.19-16 (2026-03-13 11:18:39)
```

```r
library(lmerTest)
```

```
## Loading required package: lme4
```

```
## Loading required package: Matrix
```

```
##
## Attaching package: 'lme4'
```

```
## The following object is masked from 'package:mice':
##
##     toenail
```

```
##
## Attaching package: 'lmerTest'
```

```
## The following object is masked from 'package:lme4':
##
##     lmer
```

```
## The following object is masked from 'package:stats':
##
##     step

library(ggplot2)
library(lavaan)

## This is lavaan 0.6-21
## lavaan is FREE software! Please report any bugs.

library(semPlot)
library(cocor)
library(car)

## Loading required package: carData

## Registered S3 method overwritten by 'car':
##   method           from
##   na.action.merMod lme4

##
## Attaching package: 'car'

## The following object is masked from 'package:dplyr':
##
##     recode

library(corrplot)

## corrplot 0.95 loaded

library(semptools)
library(effectsize)
library(gridExtra)

##
## Attaching package: 'gridExtra'

## The following object is masked from 'package:dplyr':
##
##     combine

library(gamm4)

## Loading required package: mgcv

## Loading required package: nlme

##
## Attaching package: 'nlme'

## The following object is masked from 'package:lme4':
##
##     lmList
```

```
## The following object is masked from 'package:dplyr':
##
##     collapse

## This is mgcv 1.9-4. For overview type '?mgcv'.

## This is gamm4 0.2-7

library(tidyr)

##
## Attaching package: 'tidyr'

## The following objects are masked from 'package:Matrix':
##
##     expand, pack, unpack

library(gt)
library(tidyverse)

## ── Attaching core tidyverse packages ──────────────────────── tidyverse
2.0.0 ──
## ✓ forcats   1.0.1     ✓ readr     2.2.0
## ✓ lubridate 1.9.5     ✓ stringr   1.6.0
## ✓ purrr     1.2.2     ✓ tibble    3.3.1

## ── Conflicts ────────────────────────────────────────
tidyverse_conflicts() ──
## ✖ nlme::collapse()     masks dplyr::collapse()
## ✖ gridExtra::combine() masks dplyr::combine()
## ✖ tidyr::expand()      masks Matrix::expand()
## ✖ mice::filter()       masks dplyr::filter(), stats::filter()
## ✖ dplyr::lag()         masks stats::lag()
## ✖ tidyr::pack()        masks Matrix::pack()
## ✖ car::recode()        masks dplyr::recode()
## ✖ purrr::some()        masks car::some()
## ✖ tidyr::unpack()      masks Matrix::unpack()
## ℹ Use the conflicted package (<http://conflicted.r-lib.org/>) to force
all conflicts to become errors

library(ggExtra)
library(flextable)

##
## Attaching package: 'flextable'
##
## The following object is masked from 'package:purrr':
##
##     compose

library(gridGraphics)
```

```
## Loading required package: grid

library(partR2)

plotit <- function(data, x_var, y_var, x_label, y_label, ymin, ymax,
color_choice) {
  ggplot(data, aes(x = .data[[x_var]], y = .data[[y_var]])) +
    geom_point(size = 2, stroke = 1, alpha = 0.3, color = color_choice) +
    geom_line(aes(group = Med_ID), lty = 2, alpha = 0.1, color = "black") +
    geom_smooth(method = "lm", se = FALSE, color = color_choice, size = 0.5,
lty = 1, aes(group = Med_ID)) +
    labs(x = x_label, y = y_label) +
    ylim(ymin, ymax) +
    theme_minimal() +
    theme(
      plot.title = element_text(hjust = 0.5),
      axis.title.x = element_text(size = 46),
      axis.title.y = element_text(size = 46),
      axis.text.x = element_text(size = 32),
      axis.text.y = element_text(size = 32)
    )
}
plotit2 <- function(data, x_var, y_var, x_label, y_label, ymin, ymax,
color_choice, n_highlight = 131, seed = 716, xmin = 40, xmax = 100, xbreak =
20) {
  set.seed(seed)
  highlight_ids <- sample(unique(data$Med_ID), n_highlight)
  data$highlight <- data$Med_ID %in% highlight_ids
  ggplot(data, aes(x = .data[[x_var]], y = .data[[y_var]])) +
    geom_line(data = subset(data, !highlight), aes(group = Med_ID), alpha =
0.7, linewidth = 0.3, lty = 2, color = "grey") +
    geom_point(data = subset(data, !highlight), aes(group = Med_ID), alpha =
0.3, size = 1, color = color_choice) +
    geom_line(data = subset(data, highlight), aes(group = Med_ID), alpha =
0.7, linewidth = 0.5, lty = 2, color = "grey") +
    geom_point(data = subset(data, highlight), aes(group = Med_ID), alpha =
0.3, size = 1.5, color = color_choice) +
    geom_smooth(data = subset(data, highlight), aes(group = Med_ID), method =
"lm", se = FALSE, color = color_choice, linewidth = 0.5, lty = 1) +
    geom_vline(xintercept = 50, linetype = "dashed", linewidth = 0.6, alpha =
0.7) +
    labs(x = x_label, y = y_label) +
    scale_x_continuous(
      limits = c(xmin, xmax),
      breaks = seq(xmin, xmax, by = xbreak)
    ) +
    coord_cartesian(ylim = c(ymin, ymax)) +
    scale_y_continuous(breaks = pretty(c(ymin, ymax))) +
    theme_minimal() +
    theme(
```

```
      axis.title.x = element_text(size = 38),
      axis.title.y = element_text(size = 38),
      axis.text = element_text(size = 24),
      plot.margin = margin(t = 10, r = 10, b = 10, l = 60)
    )
}
plotcross <- function(data, x_var, y_var, x_label, y_label, ymin, ymax,
color_choice) {
  ggplot(data, aes(x = .data[[x_var]], y = .data[[y_var]])) +
    geom_point(size = 0.7, stroke = 1, alpha = 1, color = color_choice) +
    geom_smooth(method = "lm", se = FALSE, color = "black", linewidth = 1,
lty = 1) +
    # geom_smooth(method = "lm", formula = y ~ poly(x, 2), se = FALSE, color
= "black", linewidth = 1, lty = 1) +
    labs(x = x_label, y = y_label) +
    ylim(ymin, ymax) +
    theme_minimal() +
    theme(
      plot.title = element_text(hjust = 0.5),
      axis.title.x = element_text(size = 32),
      axis.title.y = element_text(size = 32),
      axis.text.x = element_text(size = 28),
      axis.text.y = element_text(size = 28)
    )
}
plot_outliers <- function(data, x_var, y_var, x_label, y_label, ymin, ymax,
outliers, color_choice) {
  outlier_CCIDs <- unique(outliers$Med_ID)
  data <- data %>%
    mutate(is_outlier = ifelse(Med_ID %in% outlier_CCIDs, TRUE, FALSE))
  non_outlier_data <- data %>%
    filter(!is_outlier)
  ggplot(data, aes(x = .data[[x_var]], y = .data[[y_var]])) +
    geom_point(aes(color = is_outlier, alpha = is_outlier), size = 2, stroke
= 1) +
    geom_line(aes(group = Med_ID), lty = 2, alpha = 1, color = "black") +
    geom_smooth(
      data = non_outlier_data,
      aes(x = .data[[x_var]], y = .data[[y_var]]),
      method = "lm", se = FALSE, size = 0.8, lty = 1, color = "black"
    ) +
    labs(x = x_label, y = y_label) +
    ylim(ymin, ymax) +
    scale_color_manual(
      values = c("TRUE" = "green", "FALSE" = color_choice),
      labels = c("TRUE" = "Outlier", "FALSE" = "Non-Outlier")
    ) +
    scale_alpha_manual(values = c("TRUE" = 1, "FALSE" = 1), guide = "none") +
    theme_minimal() +
    theme(
```

```
      plot.title = element_text(hjust = 0.5),
      legend.position = "bottom",
      legend.box = "horizontal",
      legend.text = element_text(size = 12),
      axis.title.x = element_text(size = 24),
      axis.title.y = element_text(size = 24),
      axis.text.x = element_text(size = 16),
      axis.text.y = element_text(size = 16)
    ) +
    guides(color = guide_legend(title = NULL))
}
label_mahalanobis_outliers <- function(data, id_col, variables, flag_col_name
= "outlier_flag") {
  data_matrix <- data[, variables]
  dists <- mahalanobis(
    data_matrix,
    colMeans(data_matrix, na.rm = TRUE),
    cov(data_matrix, use = "complete.obs")
  )
  is_outlier <- dists > qchisq(0.975, df = length(variables))
  outlier_ids <- data[[id_col]][is_outlier]
  data[[flag_col_name]] <- ifelse(data[[id_col]] %in% outlier_ids, "Outlier",
"Normal")
  return(data)
}

brain_data <-
read.csv("~/ownCloud/GM_revision/HABS_HD/COMBAT_data/HABS_data_longCombatCorr
ectedROIGLOBTOGETHER.csv")
cog_data   <-
read.csv("~/ownCloud/GM_revision/HABS_HD/COG_data/HABS_gCog_data_g_derived.cs
v")

mydata     <- merge(brain_data, cog_data, by = c("Med_ID", "Visit_ID"))

cols <- gsub("\\.x$", "", grep("\\.x$", names(mydata), value = TRUE))

for (cn in cols) {
  y_col <- paste0(cn, ".y")
  if (y_col %in% names(mydata)) mydata[[y_col]] <- NULL
  x_col <- paste0(cn, ".x")
  if (x_col %in% names(mydata)) names(mydata)[names(mydata) == x_col] <- cn
}
```

#Pattern-mixture allocation

```
subject_patterns <- mydata %>%
  group_by(Med_ID) %>%
  summarize(
    wave0 = as.integer(any(Visit_ID == 1)),
```

```
    wave1 = as.integer(any(Visit_ID == 2)),
    wave2 = as.integer(any(Visit_ID == 3)),
    wave3 = as.integer(any(Visit_ID == 4)),
    .groups = "drop"
  ) %>%
  mutate(
    pattern = paste0(wave0, wave1, wave2, wave3),
    pattern_group = case_when(
      pattern %in% c("1110", "1010", "1100") ~ "G1",
      pattern %in% c("1111", "1101", "1001", "1011") ~ "G2",
      TRUE ~ "Other"
    )
  )

subject_patterns %>% count(pattern) %>% arrange(desc(n))
```

```
## # A tibble: 8 × 2
##   pattern     n
##   <chr>   <int>
## 1 1000     1499
## 2 1100      759
## 3 1110      448
## 4 1111      106
## 5 0100        5
## 6 0111        2
## 7 1010        1
## 8 1101        1
```

```
mydata <- mydata %>% left_join(subject_patterns %>% select(Med_ID,
pattern_group, pattern), by = "Med_ID")
sapply(split(mydata$Med_ID, mydata$pattern_group), function(x)
length(unique(x)))
```

```
##    G1    G2 Other
##  1208   107  1506
```

```
summary_table <- aggregate(dA ~ pattern_group, data = mydata,
                           FUN = function(x) c(count = length(x),
                                               mean = mean(x, na.rm = TRUE),
                                               min = min(x, na.rm = TRUE),
                                               max = max(x, na.rm = TRUE)))

summary_table <- do.call(data.frame, summary_table)
names(summary_table) <- c("pattern_group", "count", "mean_dA", "min_dA",
"max_dA")
summary_table
```

```
##   pattern_group count    mean_dA min_dA max_dA
## 1            G1  2864 1.69988827      0   6.67
## 2            G2   427 3.25365340      0   7.00
## 3         Other  1510 0.01060927      0   5.00
```

```
names(mydata) <- gsub("mean_MeanThickness_thickness.combat", "Thickness",
names(mydata))
names(mydata) <- gsub("mean_WhiteSurfArea_area.combat", "Area",
names(mydata))
names(mydata) <- gsub("mean_Volume.combat", "Volume", names(mydata))
```

CROSS-SECTIONAL ANALYSIS

#Descriptive statistic of the sample

```
mydata_cross <- mydata[mydata$Visit_ID == 1, ]
mydata_cross <- mydata[mydata$Visit_ID == 1 & mydata$CS == "CU", ]

length(unique(mydata_cross$Med_ID))

## [1] 2814

length(unique(mydata_cross$Med_ID))

## [1] 2814

length(unique(mydata_cross$Med_ID))

## [1] 2814

cat("Cross-sectional age summary (A0_raw):\n")

## Cross-sectional age summary (A0_raw):

cat("Cross-sectional sex:\n")

## Cross-sectional sex:

print(table(factor(mydata_cross$Sex, levels = c(1, 0), labels = c("Female",
"Male"))))

##
## Female   Male
##   1851    963

print(table(factor(mydata[mydata$Visit_ID == 1, ]$Ethnicity, levels =
c("Black", "Hispanic", "White", " "), labels = c("Black", "Hispanic",
"White", "unknown"))))

##
##    Black Hispanic    White  unknown
##      685     1055     1074        0
```

Mediation models

Outliers

```
mydata_cross <- label_mahalanobis_outliers(
  data = mydata_cross,
  id_col = "Med_ID",
  variables = c("gCog0", "Thickness", "Area", "A0", "Sex", "YoE"),
  flag_col_name = "outlier_flag"
)
all_outliers_cross <- data.frame(Med_ID =
mydata_cross$Med_ID[mydata_cross$outlier_flag == "Outlier"])

mydata_cross[mydata_cross$Med_ID %in% all_outliers_cross$Med_ID,
setdiff(names(mydata_cross), "Med_ID")] <- NA
mydata_cross <- mydata_cross[!is.na(mydata_cross$gCog0), ]
mydata_cross$gCog <- mydata_cross$gCog0

vars_to_scale <- c("Age", "Sex", "YoE", "Thickness", "Area", "gCog")
mydata_cross[vars_to_scale] <- scale(mydata_cross[vars_to_scale])
```

Correlation matrix (final dataset)

```
corr_vars <- mydata_cross[, c("Age", "gCog", "Thickness", "Area", "Sex")]
colnames(corr_vars) <- c("Age","COG (g)", "THK", "SA", "Sex")
matrix    <- cor(corr_vars)
testRes   <- cor.mtest(corr_vars, conf.level = 0.95)
png("Figures/corrmat_withoutYOE_HABS.png", width = 6, height = 6, units =
"in", res = 300)
cormat    <- corrplot(matrix, method = 'number', p.mat = testRes$p, sig.level
= 0.05,
                      order = 'original', type = "lower", diag = FALSE,
tl.col = "black")
dev.off()

## quartz_off_screen
##                 2

corr_vars <- mydata_cross[, c("Age", "gCog", "Thickness", "Area", "Sex",
"YoE")]
colnames(corr_vars) <- c("Age","COG (g)", "THK", "SA", "Sex", "YoE")
matrix    <- cor(corr_vars)
testRes   <- cor.mtest(corr_vars, conf.level = 0.95)
png("Figures/corrmat_withYOE_HABS.png", width = 6, height = 6, units = "in",
res = 300)
cormat    <- corrplot(matrix, method = 'number', p.mat = testRes$p, sig.level
= 0.05,
                      order = 'original', type = "lower", diag = FALSE,
tl.col = "black")
dev.off()
```

```
## quartz_off_screen
##                 2

vars_to_scale <- c("Age", "Sex", "YoE", "Thickness", "Area", "gCog")
mydata_cross[vars_to_scale] <- scale(mydata_cross[vars_to_scale])

SEM1 <- '
  gCog  ~ c1*Age + b1*Thickness + b2*Area + Sex

  Thickness  ~ a1*Age + Sex
  Area   ~ a2*Age  + Sex

  Thickness ~~ Area

  ATC := abs(a1 * b1)
  ASC := abs(a2 * b2)
  ABC := ATC + ASC

  Age_total := ABC + abs(c1)

  ATC_prop := ATC / Age_total
  ASC_prop := ASC / Age_total

  agecontrast := ATC_prop - ASC_prop

'
set.seed(456)
# SEM_fit1 <- sem(SEM1, data = mydata_cross, se = "boot", bootstrap = 5000)
# summary(SEM_fit1, fit.measures=TRUE, std = TRUE)

SEM1_TIV <- '
  gCog  ~ c1*Age + b1*Thickness + b2*Area + Sex

  Thickness  ~ a1*Age + eTIV + Sex
  Area   ~ a2*Age  + eTIV + Sex

  Thickness ~~ Area

  ATC := abs(a1 * b1)
  ASC := abs(a2 * b2)
  ABC := ATC + ASC

  Age_total := ABC + abs(c1)

  ATC_prop := ATC / Age_total
  ASC_prop := ASC / Age_total

'
```

```r
set.seed(456)
SEM_fit1_TIV <- sem(SEM1_TIV, data = mydata_cross, se = "boot", bootstrap = 5000)
summary(SEM_fit1_TIV, fit.measures=TRUE, std = TRUE)
```

```
## lavaan 0.6-21 ended normally after 9 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        14
##
##   Number of observations                          2749
##
## Model Test User Model:
##
##   Test statistic                                21.385
##   Degrees of freedom                                 1
##   P-value (Chi-square)                           0.000
##
## Model Test Baseline Model:
##
##   Test statistic                              4614.855
##   Degrees of freedom                                12
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.996
##   Tucker-Lewis Index (TLI)                       0.947
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -9403.751
##   Loglikelihood unrestricted model (H1)      -9393.058
##
##   Akaike (AIC)                               18835.502
##   Bayesian (BIC)                             18918.368
##   Sample-size adjusted Bayesian (SABIC)      18873.885
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.086
##   90 Percent confidence interval - lower         0.057
##   90 Percent confidence interval - upper         0.120
##   P-value H_0: RMSEA <= 0.050                    0.023
##   P-value H_0: RMSEA >= 0.080                    0.666
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.008
```

```
##
## Parameter Estimates:
##
##   Standard errors                             Bootstrap
##   Number of requested bootstrap draws             5000
##   Number of successful bootstrap draws            5000
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   gCog ~
##     Age       (c1)   -0.142    0.019   -7.642    0.000   -0.142   -0.142
##     Thickness (b1)    0.135    0.018    7.393    0.000    0.135    0.135
##     Area      (b2)    0.326    0.020   16.265    0.000    0.326    0.326
##     Sex               0.163    0.021    7.890    0.000    0.163    0.163
##   Thickness ~
##     Age       (a1)   -0.310    0.018  -17.413    0.000   -0.310   -0.310
##     eTIV              0.116    0.023    5.128    0.000    0.116    0.115
##     Sex               0.104    0.022    4.693    0.000    0.104    0.104
##   Area ~
##     Age       (a2)   -0.160    0.010  -16.388    0.000   -0.160   -0.160
##     eTIV              0.837    0.014   61.640    0.000    0.837    0.828
##     Sex              -0.058    0.013   -4.618    0.000   -0.058   -0.058
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Thickness ~~
##    .Area             -0.135    0.010  -13.558    0.000   -0.135   -0.279
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .gCog              0.872    0.023   37.583    0.000    0.872    0.872
##    .Thickness         0.898    0.023   38.365    0.000    0.898    0.899
##    .Area              0.260    0.008   31.150    0.000    0.260    0.260
##
## Defined Parameters:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##     ATC               0.042    0.006    6.929    0.000    0.042    0.042
##     ASC               0.052    0.004   11.624    0.000    0.052    0.052
##     ABC               0.094    0.008   12.493    0.000    0.094    0.094
##     Age_total         0.236    0.018   13.343    0.000    0.236    0.236
##     ATC_prop          0.178    0.029    6.135    0.000    0.178    0.178
##     ASC_prop          0.222    0.024    9.326    0.000    0.222    0.222
```

```
SEM1_YoE <- '
  gCog  ~ c1*Age + b1*Thickness + b2*Area + Sex + YoE

  Thickness  ~ a1*Age + Sex
  Area   ~ a2*Age  + Sex

  Thickness ~~ Area
```

```
  YoE ~ Area

  ATC := abs(a1 * b1)
  ASC := abs(a2 * b2)
  ABC := ATC + ASC

  Age_total := ABC + abs(c1)

  ATC_prop := ATC / Age_total
  ASC_prop := ASC / Age_total

'
set.seed(456)
SEM_fit1_YoE <- sem(SEM1_YoE, data = mydata_cross, se = "boot", bootstrap =
5000)
summary(SEM_fit1_YoE, fit.measures=TRUE, std = TRUE)

## lavaan 0.6-21 ended normally after 8 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        15
##
##   Number of observations                          2749
##
## Model Test User Model:
##
##   Test statistic                                59.433
##   Degrees of freedom                                 3
##   P-value (Chi-square)                           0.000
##
## Model Test Baseline Model:
##
##   Test statistic                              3303.646
##   Degrees of freedom                                14
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.983
##   Tucker-Lewis Index (TLI)                       0.920
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)             -13978.541
##   Loglikelihood unrestricted model (H1)     -13948.825
##
##   Akaike (AIC)                               27987.082
```

```
##   Bayesian (BIC)                                 28075.867
##   Sample-size adjusted Bayesian (SABIC)          28028.207
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                             0.083
##   90 Percent confidence interval - lower            0.065
##   90 Percent confidence interval - upper            0.102
##   P-value H_0: RMSEA <= 0.050                       0.001
##   P-value H_0: RMSEA >= 0.080                       0.622
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                              0.031
##
## Parameter Estimates:
##
##   Standard errors                                Bootstrap
##   Number of requested bootstrap draws                 5000
##   Number of successful bootstrap draws                5000
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   gCog ~
##     Age       (c1)   -0.226    0.015  -15.075    0.000   -0.226   -0.224
##     Thickness (b1)    0.072    0.015    4.939    0.000    0.072    0.072
##     Area      (b2)    0.189    0.016   11.906    0.000    0.189    0.187
##     Sex               0.144    0.016    8.916    0.000    0.144    0.143
##     YoE               0.613    0.013   47.226    0.000    0.613    0.607
##   Thickness ~
##     Age       (a1)   -0.301    0.018  -16.945    0.000   -0.301   -0.301
##     Sex               0.035    0.018    1.887    0.059    0.035    0.035
##   Area ~
##     Age       (a2)   -0.095    0.016   -6.086    0.000   -0.095   -0.095
##     Sex              -0.559    0.017  -33.804    0.000   -0.559   -0.559
##   YoE ~
##     Area              0.190    0.018   10.534    0.000    0.190    0.190
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##  .Thickness ~~
##    .Area             -0.076    0.015   -5.108    0.000   -0.076   -0.096
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .gCog              0.517    0.014   37.078    0.000    0.517    0.506
##    .Thickness         0.906    0.024   38.278    0.000    0.906    0.907
##    .Area              0.686    0.019   36.539    0.000    0.686    0.686
##    .YoE               0.963    0.027   35.399    0.000    0.963    0.964
##
```

```
## Defined Parameters:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##     ATC               0.022    0.005    4.774    0.000    0.022    0.022
##     ASC               0.018    0.003    5.468    0.000    0.018    0.018
##     ABC               0.040    0.006    6.977    0.000    0.040    0.039
##     Age_total         0.266    0.014   18.466    0.000    0.266    0.263
##     ATC_prop          0.082    0.018    4.574    0.000    0.082    0.082
##     ASC_prop          0.067    0.012    5.516    0.000    0.067    0.067
```

LONGITUDINAL ANALYSIS

Derive slopes - uncomment if you want to recalculate slopes. This might take ~15-20 min.

```
mydata_long <- subset(mydata, pattern_group %in% c("G1", "G2"))
mydata_long <- mydata_long[mydata_long$CS == "CU", ]
mydata_long$Visit_ID <- as.numeric(as.character(mydata_long$Visit_ID))

# global_vars <- c("Thickness", "Area", "Volume")
#
# roi_thk_vars  <- grep("^mean_.*_thickness.combat$", names(mydata_long),
value = TRUE)
# roi_area_vars <- grep("^mean_.*_area.combat$",      names(mydata_long),
value = TRUE)
#
# roi_vars <- c(roi_thk_vars, roi_area_vars)
# all_vars <- c(global_vars, roi_vars)
#
# for (v in all_vars) {
#
#   mydata_long[[paste0(v, "_A0")]] <-
#     residuals(lmer(
#       as.formula(paste0(v, " ~ 1 + (1|Med_ID)")),
#       data = mydata_long, REML = TRUE
#     ))
#
#   mydata_long[[paste0(v, "_A0dA")]] <-
#     residuals(lmer(
#       as.formula(paste0(v, " ~ A0:dA + (1|Med_ID)")),
#       data = mydata_long, REML = TRUE
#     ))
#
#   mydata_long[[paste0(v, "_gamm")]] <-
#     residuals(
#       gamm(
#         as.formula(paste0(v, " ~ s(Age)")),
#         data = mydata_long,
#         random = list(Med_ID = ~1),
#         REML = TRUE
#       )$gam,
#       type = "response"
#     )
```

```
# }
#
# split_by_id <- split(mydata_long, mydata_long$Med_ID)
# split_by_id <- split_by_id[sapply(split_by_id, nrow) > 1]
#
# slope <- function(df, y) {
#   as.numeric(coef(lm(as.formula(paste0(y, " ~ dA")), data = df))["dA"])
# }
#
# get_visits <- function(df, var) {
#   out <- rep(NA_real_, 4)
#   idx <- match(1:4, df$Visit_ID)
#   out[!is.na(idx)] <- df[[var]][idx[!is.na(idx)]]
#   out
# }
#
# results <- lapply(names(split_by_id), function(sid) {
#   d  <- split_by_id[[sid]]
#   w0 <- d[which.min(d$dA), ]
#
#   global_slopes <- c(
#     gCog_slope_A0    = slope(d, "gCog_A0"),
#     gCog_slope_A0dA  = slope(d, "gCog_A0dA"),
#     gCog_slope_gamm  = slope(d, "gCog_gamm"),
#
#     Thickness_slope_A0    = slope(d, "Thickness_A0"),
#     Thickness_slope_A0dA  = slope(d, "Thickness_A0dA"),
#     Thickness_slope_gamm  = slope(d, "Thickness_gamm"),
#
#     Area_slope_A0    = slope(d, "Area_A0"),
#     Area_slope_A0dA  = slope(d, "Area_A0dA"),
#     Area_slope_gamm  = slope(d, "Area_gamm"),
#
#     Volume_slope_A0    = slope(d, "Volume_A0"),
#     Volume_slope_A0dA  = slope(d, "Volume_A0dA"),
#     Volume_slope_gamm  = slope(d, "Volume_gamm")
#   )
#   roi_slopes <- unlist(lapply(roi_vars, function(v) {
#     name <- sub("^mean_(.*)\\.combat$", "\\1", v)
#     c(
#       setNames(slope(d, paste0(v, "_A0")),    paste0(name, "_slope_A0")),
#       setNames(slope(d, paste0(v, "_A0dA")),  paste0(name, "_slope_A0dA")),
#       setNames(slope(d, paste0(v, "_gamm")),  paste0(name, "_slope_gamm"))
#     )
#   }))
# data.frame(
#   Med_ID = sid,
#
#   as.list(global_slopes),
#   as.list(roi_slopes),
```

```
#
#   Thickness_baseline = w0$Thickness,
#   Area_baseline      = w0$Area,
#   gCog_baseline      = w0$Volume,
#
#   Sex = w0$Sex,
#   YoE = w0$YoE,
#   A0  = w0$A0,
#
#   as.list(setNames(get_visits(d, "Thickness"), paste0("Thickness", 0:3))),
#   as.list(setNames(get_visits(d, "Area"), paste0("Area", 0:3))),
#   as.list(setNames(get_visits(d, "gCog0"),          paste0("gCog0", 0:3))),
#
#   pattern_group = w0$pattern_group,
#   Ethnicity     = w0$Ethnicity,
#   CS            = w0$CS
# )
# })
# mydata_slopes <- do.call(rbind, results)
#
write.csv(mydata_slopes,"COG_data/HABS_brain_cog_slopesnonharmROIGLOB.csv",ro
w.names = FALSE)
```

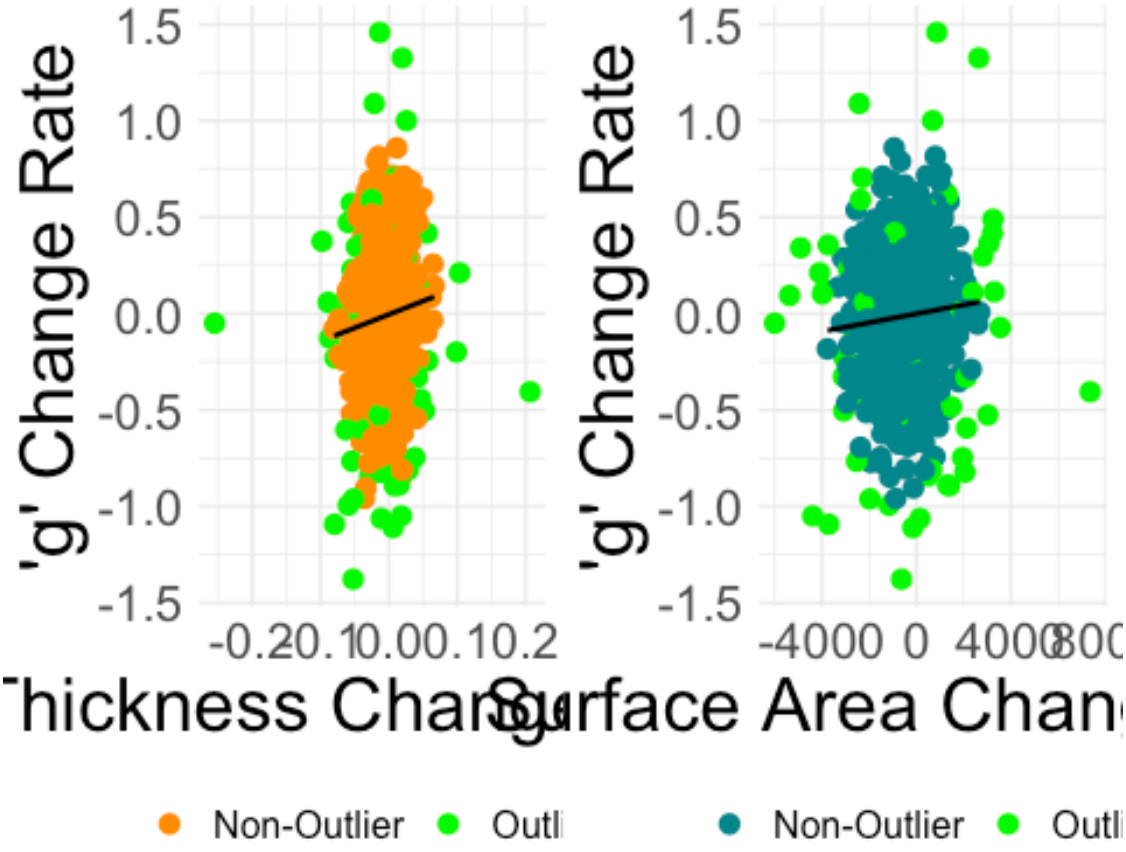
Outliers

```
mydata_slopes <- read.csv("COG_data/HABS_brain_cog_slopesnonharmROIGLOB.csv")
mydata_slopes <- mydata_slopes[mydata_slopes$CS == "CU", ]
# mydata <- mydata[mydata$Med_ID %in% mydata_slopes$Med_ID, ]

# plotcross(mydata_slopes, "Thickness_slope_A0", "gCog_slope_A0", "GM
Thickness Change Rate", "Cognition Change Rate",
min(mydata_slopes$gCog_slope_A0), max(mydata_slopes$gCog_slope_A0),
color_choice = "darkorange")
# plotcross(mydata_slopes, "Area_slope_A0", "gCog_slope_A0", "GM Area Change
Rate", "Cognition Change Rate", min(mydata_slopes$gCog_slope_A0),
max(mydata_slopes$gCog_slope_A0),  color_choice = "turquoise4")

mydata_slopes <- label_mahalanobis_outliers(data = mydata_slopes, id_col =
"Med_ID", variables = c("Thickness_slope_A0", "Area_slope_A0", "Sex", "YoE",
"gCog_slope_A0", "A0"), flag_col_name = "outlier_flag_thick")

all_outliers <- data.frame(Med_ID =
mydata_slopes$Med_ID[mydata_slopes$outlier_flag_thick == "Outlier"])

outlier_thickness_plot <- plot_outliers(mydata_slopes, "Thickness_slope_A0",
"gCog_slope_A0",
                                        "Thickness Change Rate", "'g' Change
Rate",
                                         min(mydata_slopes$gCog_slope_A0),
                                         max(mydata_slopes$gCog_slope_A0),
                                         all_outliers, "darkorange")
```

```
## Warning: Using `size` aesthetic for lines was deprecated in ggplot2 3.4.0.
## ℹ Please use `linewidth` instead.
## This warning is displayed once per session.
## Call `lifecycle::last_lifecycle_warnings()` to see where this warning was
## generated.
```

```
outlier_area_plot      <- plot_outliers(mydata_slopes, "Area_slope_A0",
"gCog_slope_A0",
                                        "Surface Area Change Rate", "'g'
Change Rate",
                                        min(mydata_slopes$gCog_slope_A0),
                                        max(mydata_slopes$gCog_slope_A0),
                                        all_outliers, "turquoise4")

grid.arrange(outlier_thickness_plot, outlier_area_plot, ncol = 2)
```

```
## `geom_smooth()` using formula = 'y ~ x'
## `geom_line()`: Each group consists of only one observation.
## ℹ Do you need to adjust the group aesthetic?
## `geom_smooth()` using formula = 'y ~ x'
## `geom_line()`: Each group consists of only one observation.
## ℹ Do you need to adjust the group aesthetic?
```



```r
mydata_slopes[mydata_slopes$Med_ID %in% all_outliers$Med_ID,
setdiff(names(mydata_slopes), "Med_ID")] <- NA
HABS_slopes <- mydata_slopes
slope_cols <- grepl("slope", names(HABS_slopes))
HABS_slopes[slope_cols] <- scale(HABS_slopes[slope_cols])

thickness_plot <- plotcross(HABS_slopes, "Thickness_slope_A0",
"gCog_slope_A0", "Thickness change rate", expression(italic(g)~"change
rate"), min(HABS_slopes$gCog_slope_A0), max(HABS_slopes$gCog_slope_A0),
color_choice = "darkorange")
area_plot <- plotcross(HABS_slopes, "Area_slope_A0", "gCog_slope_A0", "Area
change rate", expression(italic(g)~"change rate"),
min(HABS_slopes$gCog_slope_A0), max(HABS_slopes$gCog_slope_A0),  color_choice
= "turquoise4")

png("Figures/Thickness_g_HABS.png", width = 8, height = 8, units = "in", res
= 300)
thickness_plot

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).

## Warning: Removed 63 rows containing missing values or values outside the
scale range
## (`geom_point()`).

dev.off()

## quartz_off_screen
##                 2

png("Figures/Area_g_HABS.png", width = 8, height = 8, units = "in", res =
300)
area_plot

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 63 rows containing missing values or values outside the scale
range
## (`geom_point()`).

dev.off()

## quartz_off_screen
##                 2
```

```
png("Figures/THK_SA_COG_corr_HABS.png", width = 8, height = 16, units = "in",
res = 300)
grid.arrange(thickness_plot, area_plot, ncol = 1)

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 63 rows containing missing values or values outside the scale
range
## (`geom_point()`).

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 63 rows containing missing values or values outside the scale
range
## (`geom_point()`).

dev.off()

## quartz_off_screen
##                 2

png("Figures/THK_SA_COG_corr_HABS_wide.png", width = 16, height = 8, units =
"in", res = 300)
grid.arrange(thickness_plot, area_plot, ncol = 2)

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 63 rows containing missing values or values outside the scale
range
## (`geom_point()`).

## `geom_smooth()` using formula = 'y ~ x'

## Warning: Removed 63 rows containing non-finite outside the scale range
## (`stat_smooth()`).
## Removed 63 rows containing missing values or values outside the scale
range
## (`geom_point()`).

dev.off()

## quartz_off_screen
##                 2
```

##Univariate mixed-effects models

```r
mydata_long <- mydata_long[!mydata_long$Med_ID %in% all_outliers$Med_ID, ]

Interaction <- (mydata_long$dA - mean(mydata_long$dA)) * (mydata_long$A0 -
mean(mydata_long$A0))
OrthInteraction <- residuals(lmer(Interaction ~ dA + A0 + (1|Med_ID), data =
mydata_long))
```

```
## boundary (singular) fit: see help('isSingular')
```

```r
# thickness_m <- lmer(Thickness ~ A0 + dA + MRI_Scanner + Ethnicity +
OrthInteraction + (1|Med_ID), data=mydata_long)
# area_m      <- lmer(Area      ~ A0 + dA + MRI_Scanner + Ethnicity +
OrthInteraction + (1|Med_ID), data=mydata_long)
#
# summary(thickness_m)
# summary(area_m)

thickness_m <- lmer(Thickness ~ A0 + dA  + OrthInteraction + (1|Med_ID),
data=mydata_long)
area_m      <- lmer(Area      ~ A0 + dA  + OrthInteraction + (1|Med_ID),
data=mydata_long)
volume_m    <- lmer(Volume    ~ A0 + dA  + OrthInteraction + (1|Med_ID),
data=mydata_long)

summary(thickness_m)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Thickness ~ A0 + dA + OrthInteraction + (1 | Med_ID)
##    Data: mydata_long
##
## REML criterion at convergence: -8748.7
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.7608 -0.4849  0.0098  0.5272  3.1680
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  Med_ID   (Intercept) 0.004774 0.06909
##  Residual             0.001549 0.03935
## Number of obs: 3164, groups:  Med_ID, 1252
##
## Fixed effects:
##                    Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)      2.607e+00  1.726e-02  1.259e+03 151.035   <2e-16 ***
## A0              -3.318e-03  2.614e-04  1.255e+03 -12.689   <2e-16 ***
## dA              -4.700e-03  3.921e-04  1.994e+03 -11.986   <2e-16 ***
## OrthInteraction -1.246e-04  4.970e-05  1.993e+03  -2.508   0.0122 *
## ---
```

```
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) A0     dA
## A0          -0.992
## dA          -0.041  0.001
## OrthIntrctn -0.029  0.030 -0.015
```

```
summary(area_m)
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Area ~ A0 + dA + OrthInteraction + (1 | Med_ID)
##    Data: mydata_long
##
## REML criterion at convergence: 62764.4
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.6398 -0.4560 -0.0064  0.4566  6.7017
##
## Random effects:
##  Groups   Name        Variance  Std.Dev.
##  Med_ID   (Intercept) 232294297 15241
##  Residual               3096508  1760
## Number of obs: 3164, groups:  Med_ID, 1252
##
## Fixed effects:
##                   Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)     170120.784   3578.292   1250.418  47.542  < 2e-16 ***
## A0                -100.849     54.254   1250.219  -1.859   0.0633 .
## dA                -477.634     17.706   1913.706 -26.976  < 2e-16 ***
## OrthInteraction    -11.083      2.244   1913.636  -4.939 8.52e-07 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) A0     dA
## A0          -0.993
## dA          -0.009  0.000
## OrthIntrctn -0.007  0.007 -0.016
```

```
standardize_parameters(thickness_m)
```

```
## # Standardization method: refit
##
## Parameter       | Std. Coef. |         95% CI
## ------------------------------------------------
## (Intercept)     |  -9.01e-03 | [-0.06,  0.04]
## A0              |      -0.31 | [-0.36, -0.26]
```

```
## dA              |      -0.11 | [-0.13, -0.09]
## OrthInteraction |      -0.02 | [-0.04,  0.00]

standardize_parameters(area_m)

## # Standardization method: refit
##
## Parameter       | Std. Coef. |         95% CI
## ------------------------------------------------
## (Intercept)     |  -6.88e-03 | [-0.06,  0.05]
## A0              |      -0.05 | [-0.11,  0.00]
## dA              |      -0.06 | [-0.06, -0.06]
## OrthInteraction |      -0.01 | [-0.02, -0.01]

summary(lmer(Thickness ~ A0 + dA*pattern_group + OrthInteraction +
(1|Med_ID), data=mydata_long))

## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Thickness ~ A0 + dA * pattern_group + OrthInteraction + (1 |
##     Med_ID)
##    Data: mydata_long
##
## REML criterion at convergence: -8731.2
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.7467 -0.4844  0.0096  0.5218  3.1904
##
## Random effects:
##  Groups   Name        Variance Std.Dev.
##  Med_ID   (Intercept) 0.004773 0.06909
##  Residual             0.001548 0.03935
## Number of obs: 3164, groups:  Med_ID, 1252
##
## Fixed effects:
##                      Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)         2.604e+00  1.740e-02  1.259e+03 149.693  < 2e-16 ***
## A0                 -3.294e-03  2.626e-04  1.256e+03 -12.547  < 2e-16 ***
## dA                 -4.447e-03  4.598e-04  1.981e+03  -9.670  < 2e-16 ***
## pattern_groupG2     1.105e-02  7.766e-03  1.457e+03   1.423  0.15491
## OrthInteraction    -1.330e-04  5.033e-05  1.992e+03  -2.643  0.00827 **
## dA:pattern_groupG2 -1.108e-03  9.073e-04  1.933e+03  -1.222  0.22204
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) A0     dA     ptt_G2 OrthIn
## A0          -0.991
## dA          -0.032 -0.011
## pttrn_grpG2 -0.123  0.084  0.098
```

```
## OrthIntrctn -0.030  0.033 -0.095 -0.030
## dA:pttrn_G2  0.013  0.009 -0.517 -0.327  0.156
```

```
summary(lmer(Area ~ A0 + dA*pattern_group + OrthInteraction + (1|Med_ID),
data=mydata_long))
```

```
## Linear mixed model fit by REML. t-tests use Satterthwaite's method [
## lmerModLmerTest]
## Formula: Area ~ A0 + dA * pattern_group + OrthInteraction + (1 | Med_ID)
##    Data: mydata_long
##
## REML criterion at convergence: 62737.4
##
## Scaled residuals:
##     Min      1Q  Median      3Q     Max
## -4.5841 -0.4530 -0.0047  0.4576  6.7635
##
## Random effects:
##  Groups   Name        Variance  Std.Dev.
##  Med_ID   (Intercept) 232423351 15245
##  Residual               3096648  1760
## Number of obs: 3164, groups:  Med_ID, 1252
##
## Fixed effects:
##                       Estimate Std. Error         df t value Pr(>|t|)
## (Intercept)        170312.194   3607.344   1249.449  47.213  < 2e-16 ***
## A0                   -103.136     54.484   1249.290  -1.893   0.0586 .
## dA                   -466.574     20.737   1912.166 -22.500  < 2e-16 ***
## pattern_groupG2      -579.767   1554.444   1258.343  -0.373   0.7092
## OrthInteraction       -11.451      2.272   1912.623  -5.039 5.12e-07 ***
## dA:pattern_groupG2    -41.134     40.676   1909.993  -1.011   0.3120
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Correlation of Fixed Effects:
##             (Intr) A0     dA     ptt_G2 OrthIn
## A0          -0.992
## dA          -0.007 -0.002
## pttrn_grpG2 -0.124  0.088  0.022
## OrthIntrctn -0.007  0.008 -0.096 -0.007
## dA:pttrn_G2  0.003  0.002 -0.520 -0.073  0.158
```

```
extract_lmer_summary <- function(model, model_name) {
  sm  <- summary(model)$coefficients
  std <- standardize_parameters(model)
  pr2 <- partR2(model, partvars = c("A0", "dA", "OrthInteraction"), R2_type =
"marginal")
  part_df <- pr2$R2 %>%
    as.data.frame() %>%
    rename(Predictor = term) %>%
```

```
    mutate(
      part_R2 = .[[ setdiff(names(.), c("Predictor", "CI_lower",
"CI_upper"))[1] ]]
    ) %>%
    select(Predictor, part_R2)

  as.data.frame(sm) %>%
    rownames_to_column("Predictor") %>%
    filter(Predictor %in% c("A0", "dA", "OrthInteraction")) %>%
    left_join(
      std %>%
        filter(Parameter %in% c("A0", "dA", "OrthInteraction")) %>%
        select(Parameter, Std_Coefficient),
      by = c("Predictor" = "Parameter")
    ) %>%
    left_join(
      part_df,
      by = "Predictor"
    ) %>%
    transmute(
      Model = model_name,
      Predictor,
      beta      = round(Estimate, 4),
      beta_std  = round(Std_Coefficient, 3),
      partR2    = round(part_R2, 3),   # unique % variance explained
      # R2_marg   = round(r2_model, 3),  # total fixed-effect variance
explained
      p         = sub("^0\\.", ".", sprintf("%.3f", `Pr(>|t|)`))
    )
}
final_table_all <- bind_rows(
  extract_lmer_summary(thickness_m, "Thickness"),
  extract_lmer_summary(area_m,      "Area"),
  extract_lmer_summary(volume_m,    "Volume"),
)
final_table_all %>% flextable() %>% set_caption(caption = "Trajectories age-
related")
```

Trajectories age-related

Model	Predictor	beta	beta_std	partR2	p
Thickness	A0	-0.0033	-0.312	0.097	.000
Thickness	dA	-0.0047	-0.108	0.011	.000
Thickness	OrthInteraction	-0.0001	-0.022	0.002	.012
Area	A0	-100.8494	-0.052	0.003	.063

Model	Predictor	beta	beta_std	partR2	p
Area	dA	-477.6336	-0.060	0.004	.000
Area	OrthInteraction	-11.0831	-0.011	0.000	.000
Volume	A0	-793.1941	-0.150	0.022	.000
Volume	dA	-2,117.6492	-0.098	0.009	.000
Volume	OrthInteraction	-38.1500	-0.014	0.001	.002

```r
set.seed(716)
# sample_ids <- sample(unique(mydata_long$Med_ID), 500)
# sample_data <- subset(mydata_long, Med_ID %in% sample_ids)

# thickness_change_plot <- plotit(mydata_long, "Age", "Thickness",
#                                 "Age", "Thickness",
#                                  min(mydata_long$Thickness),
max(mydata_long$Thickness), "darkorange")
# area_change_plot <- plotit(mydata_long, "Age", "Area",
#                                 "Age", "Area",
#                                min(mydata_long$Area),
max(mydata_long$Area), "turquoise4")

thickness_change_plot2 <- plotit2(mydata_long, "Age", "Thickness",
                                "Age", "THK (mm)",

min(mydata_long$Thickness),max(mydata_long$Thickness),"darkorange")

area_change_plot2 <- plotit2(mydata_long, "Age", "Area",
                                "Age", "SA (mm²)",

min(mydata_long$Area),max(mydata_long$Area),"turquoise4")

# png("Figures/THK_SA_age_HABS.png", width = 16, height = 8, units = "in",
res = 300)
# grid.arrange(thickness_change_plot2, area_change_plot2, ncol = 2)
# dev.off()


thickness_change_plot3 <- plotit2(mydata_long, "Age", "Thickness",
                                "Age", "Thickness (mm)",
                                2,3,"darkorange")

area_change_plot3 <- plotit2(mydata_long, "Age", "Area",
                                "Age", "Area (mm²)",
                                110000,240000,"turquoise4")

png("Figures/THK_SA_age_HABS_harmon.png", width = 16, height = 8, units =
```

```
"in", res = 300)
grid.arrange(thickness_change_plot3, area_change_plot3, ncol = 2)

## `geom_smooth()` using formula = 'y ~ x'
## `geom_smooth()` using formula = 'y ~ x'

dev.off()

## quartz_off_screen
##                 2

cog_change_plot      <- plotit2(mydata_long, "Age", "gCog0",
                                "Age", expression(italic(g)~"(z-score)"),
                                 -6, 6, "orchid4")


g1 <- ggplotGrob(cog_change_plot)

## `geom_smooth()` using formula = 'y ~ x'

g2 <- ggplotGrob(thickness_change_plot3)

## `geom_smooth()` using formula = 'y ~ x'

g3 <- ggplotGrob(area_change_plot3)

## `geom_smooth()` using formula = 'y ~ x'

left_cols <- 1:6
maxLeft <- unit.pmax(g1$widths[left_cols], g2$widths[left_cols],
g3$widths[left_cols])
g1$widths[left_cols] <- maxLeft
g2$widths[left_cols] <- maxLeft
g3$widths[left_cols] <- maxLeft

png("Figures/THK_SA_HABS.png", width = 10, height = 24, units = "in", res =
300)
grid.arrange(g1, g2, g3, ncol = 1)
dev.off()

## quartz_off_screen
##                 2

area_change_plot3

## `geom_smooth()` using formula = 'y ~ x'
```

Change-Change models

```
mydata_slopes <- mydata_slopes[mydata_slopes$CS == "CU", ] # Uncomment to
repeat analysis on healthy only
adjustments <- c("A0", "A0dA", "gamm")
```

```r
model_grid <- expand.grid(adj = adjustments,
                          measure = c("Thickness", "Area", "Volume"),
                          stringsAsFactors = FALSE)

models_simple <- lapply(seq_len(nrow(model_grid)), function(i) {
  adj <- model_grid$adj[i]
  m   <- model_grid$measure[i]
  yvar <- paste0("gCog_slope_", adj)
  xvar <- paste0(m, "_slope_", adj)
  fml <- as.formula(paste(yvar, "~", xvar))
  lm(fml, data = HABS_slopes)
})
names(models_simple) <- paste0("m_", model_grid$measure, "_", model_grid$adj)

results_simple <- do.call(
  rbind,
  lapply(names(models_simple), function(nm) {

    m  <- models_simple[[nm]]
    sm <- summary(m)$coefficients

    R2 <- rsq::rsq(m) # correct for simple lm

    data.frame(
      model     = nm,
      predictor = rownames(sm),
      beta      = sm[, "Estimate"],
      p         = sm[, "Pr(>|t|)"],
      R2      = c(NA, R2),  # intercept gets NA
      row.names = NULL
    )
  })
) %>%
  dplyr::filter(predictor != "(Intercept)")

final_table <- results_simple %>%
  mutate(
    Adjustment = case_when(
      grepl("_A0$", model)   ~ "A0",
      grepl("_A0dA$", model) ~ "A0dA",
      grepl("_gamm$", model) ~ "GAMM"
    ),
    Measure = case_when(
      grepl("^m_Thickness", model) ~ "Thickness",
      grepl("^m_Area", model)      ~ "Area",
      grepl("^m_Volume", model)    ~ "Volume",
    ),
    stat = sprintf("%.2f, R2 = %.3f, p = %.3f", beta, R2, p),
```

```
    stat = sub("p = 0\\.", "p = .", stat)
  ) %>%
  select(Adjustment, Measure, stat) %>%
  pivot_wider(names_from = Measure, values_from = stat) %>%
  arrange(factor(Adjustment, levels = c("A0", "A0dA", "GAMM")))

steiger_table <- tibble(
  Adjustment = c("A0", "A0dA", "GAMM"),
  adj_suffix = c("A0", "A0dA", "gamm")
) %>%
  rowwise() %>% mutate(
    steiger = list({
      y  <- HABS_slopes[[paste0("gCog_slope_", adj_suffix)]]
      x1 <- HABS_slopes[[paste0("Thickness_slope_", adj_suffix)]]
      x2 <- HABS_slopes[[paste0("Area_slope_", adj_suffix)]]
      ok <- complete.cases(y, x1, x2)
      ct <- cocor.dep.groups.overlap(
        r.jk = cor(y[ok], x1[ok]),
        r.jh = cor(y[ok], x2[ok]),
        r.kh = cor(x1[ok], x2[ok]),
        n    = sum(ok), alternative = "two.sided")
      tibble(
        z_steiger = sprintf("%.2f", ct@steiger1980$statistic),
        p_steiger = sub("^0\\.", ".", sprintf("%.3f",
ct@steiger1980$p.value)),
        n = sum(ok))
    })
  ) %>% unnest(steiger) %>% select(-adj_suffix)
final_table_all <- final_table %>% left_join(steiger_table, by =
"Adjustment")
final_table_all %>%
  flextable() %>%
  set_caption(
    caption = "Model Results, Thickness, Area, and Steiger Comparisons"
  ) %>%
  theme_box() %>%
  autofit()
```

Model Results, Thickness, Area, and Steiger Comparisons

Adjustment	Thickness	Area	Volume	z_steiger	p_steiger	n
A0	0.10, R2 = 0.010, p = .000	0.08, R2 = 0.006, p = .008	0.11, R2 = 0.012, p = .000	0.65	.515	1,252
A0dA	0.10, R2 = 0.010, p = .000	0.06, R2 = 0.004, p = .025	0.11, R2 = 0.011, p = .000	1.04	.297	1,252

Adjustment	Thickness	Area	Volume	z_steiger	p_steiger	n
GAMM	0.10, R2 = 0.010, p = .000	0.06, R2 = 0.003, p = .036	0.11, R2 = 0.011, p = .000	1.13	.260	1,252

ROI models

```
roi_cols <-
grep("^(?!(Thickness|Area|gCog|Volume)).*_slope_A0$",names(HABS_slopes),
                 value = TRUE, perl = TRUE)

roi_meta <- data.frame(col=roi_cols,
            region  = sub("^(.*)_(thickness|area)_slope_A0$",
"\\1",roi_cols),
            measure = sub("^.*_(thickness|area)_slope_A0$", "\\1", roi_cols),
            stringsAsFactors = FALSE)
first_three <- c("rostralmiddlefrontal", "middletemporal",
"inferiorparietal")
roi_meta <- roi_meta[order(roi_meta$region), ]
roi_meta <- rbind(roi_meta[roi_meta$region %in% first_three, ],
roi_meta[!roi_meta$region %in% first_three, ])

roi_long <- do.call(rbind, lapply(seq_len(nrow(roi_meta)), function(i) {
  cl <- roi_meta$col[i]
  msr <- roi_meta$measure[i]

  meanGM_col <- if (msr == "thickness") {
    "Thickness_slope_A0"} else {"Area_slope_A0"}

  df <- data.frame(
    ROI     = HABS_slopes[[cl]],
    MeanGM  = HABS_slopes[[meanGM_col]],
    gCog    = HABS_slopes$gCog_slope_A0
  )
  df <- df[complete.cases(df), ]
  sm <- summary(lm(gCog ~ ROI, data = df))$coefficients["ROI", ]
  data.frame(
    region  = roi_meta$region[i],
    measure = roi_meta$measure[i],
    beta    = round(sm["Estimate"], 2),
    p       = sm["Pr(>|t|)"],
    n       = nrow(df),
    stringsAsFactors = FALSE
  )
}))

roi_long$p_fdr <- ave(roi_long$p, roi_long$measure, FUN = function(x)
p.adjust(x, method = "fdr"))
```

```
roi_table <- reshape(roi_long, idvar = "region",timevar = "measure",
direction = "wide")
p_cols <- grep("^p(\\.|_fdr\\.)", names(roi_table), value = TRUE)
roi_table[p_cols] <- lapply(roi_table[p_cols],function(x) sub("^0\\.", ".",
sprintf("%.3f", x)))
roi_table %>%
  flextable() %>%
  set_caption(
    caption = "ROI Model Results"
  ) %>%
  theme_box() %>%
  fontsize(size = 9, part = "all") %>%
  autofit()
```

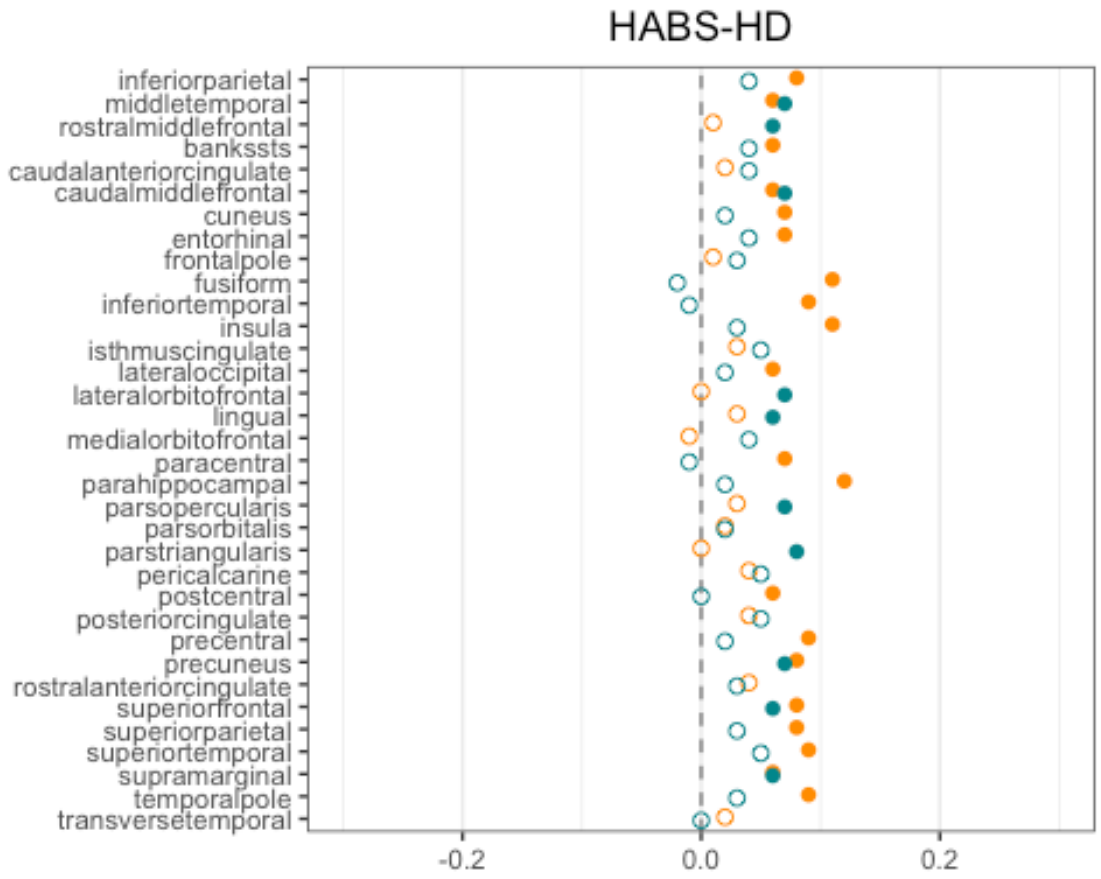
ROI Model Results

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
inferiorparietal	0.08	.005	1,252	.018	0.04	.157	1,252	.338
middletemporal	0.06	.023	1,252	.052	0.07	.014	1,252	.092
rostralmiddlefrontal	0.01	.729	1,252	.800	0.06	.023	1,252	.112
bankssts	0.06	.030	1,252	.060	0.04	.180	1,252	.360
caudalanteriorcingulate	0.02	.466	1,252	.547	0.04	.207	1,252	.370
caudalmiddlefrontal	0.06	.048	1,252	.082	0.07	.011	1,252	.092
cuneus	0.07	.013	1,252	.031	0.02	.487	1,252	.571
entorhinal	0.07	.013	1,252	.031	0.04	.199	1,252	.370
frontalpole	0.01	.668	1,252	.757	0.03	.219	1,252	.372
fusiform	0.11	.000	1,252	.002	-0.02	.415	1,252	.543
inferiortemporal	0.09	.001	1,252	.008	-0.01	.621	1,252	.681
insula	0.11	.000	1,252	.002	0.03	.316	1,252	.448
isthmuscingulate	0.03	.287	1,252	.407	0.05	.089	1,25	.233

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
							2	
lateraloccipital	0.06	.040	1,252	.072	0.02	.528	1,252	.599
lateralorbitofrontal	0.00	.910	1,252	.938	0.07	.013	1,252	.092
lingual	0.03	.300	1,252	.409	0.06	.031	1,252	.117
medialorbitofrontal	-0.01	.852	1,252	.905	0.04	.159	1,252	.338
paracentral	0.07	.010	1,252	.028	-0.01	.807	1,252	.857
parahippocampal	0.12	.000	1,252	.000	0.02	.381	1,252	.518
parsopercularis	0.03	.327	1,252	.427	0.07	.020	1,252	.112
parsorbitalis	0.02	.442	1,252	.537	0.02	.457	1,252	.571
parstriangularis	0.00	.977	1,252	.977	0.08	.004	1,252	.092
pericalcarine	0.04	.161	1,252	.245	0.05	.052	1,252	.161
postcentral	0.06	.026	1,252	.054	0.00	.883	1,252	.909
posteriorcingulate	0.04	.122	1,252	.198	0.05	.059	1,252	.168
precentral	0.09	.001	1,252	.008	0.02	.479	1,252	.571
precuneus	0.08	.003	1,252	.014	0.07	.010	1,252	.092
rostralanteriorcingulate	0.04	.166	1,252	.245	0.03	.232	1,252	.376
superiorfrontal	0.08	.008	1,252	.024	0.06	.027	1,252	.114
superiorparietal	0.08	.005	1,252	.018	0.03	.263	1,252	.406
superiortemporal	0.09	.002	1,252	.011	0.05	.103	1,252	.249

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
supramarginal	0.06	.039	1,252	.072	0.06	.044	1,252	.150
temporalpole	0.09	.001	1,252	.008	0.03	.307	1,252	.448
transversetemporal	0.02	.421	1,252	.530	0.00	.989	1,252	.989

```r
# roi_long$sig <- roi_long$p_fdr < 0.05
roi_long$sig <- roi_long$p < 0.05
roi_long$region <- factor(
  roi_long$region,
  levels = rev(unique(roi_long$region))
)
ggplot(roi_long, aes(x = beta, y = region, color = measure, shape = sig)) +
  geom_vline(xintercept = 0, linetype = "dashed", color = "grey50") +
  geom_point(position = position_dodge(width = 0.3), size = 2) +
  scale_shape_manual(values = c(`FALSE` = 1, `TRUE` = 16)) +
  scale_color_manual(values = c(thickness = "darkorange", area =
"turquoise4")) +
  scale_x_continuous(limits = c(-0.30, 0.30)) +
  labs(title = "HABS-HD", x = NULL, y = NULL) +
  theme_bw() +
  theme(panel.grid.major.y = element_blank(), legend.position = "none",
plot.title = element_text(hjust = 0.5))
```




```
# ggplot(roi_long, aes(x = beta, y = region, color = measure, shape = sig)) +
#   geom_vline(xintercept = 0, linetype = "dashed", color = "grey60") +
#   geom_point(position = position_dodge(0.6), size = 2.5) +
#   scale_color_manual(name = "Cortical measure", values = c(area =
"turquoise3", thickness = "darkorange"), labels = c(area = "SA", thickness =
"THK")) +
#   scale_shape_manual(name = "FDR-adjusted significance", values = c(`FALSE`
= 1, `TRUE` = 16), labels = c("Not significant", "q < 0.05")) +
#   theme_bw() +
#   theme(panel.grid.major.y = element_blank(), legend.position = "top",
plot.title = element_text(hjust = 0.5))

ggsave("Figures/ROI_effects_long.png", width = 3, height = 9, dpi = 300)

roi_long <- do.call(rbind, lapply(seq_len(nrow(roi_meta)), function(i) {
  cl <- roi_meta$col[i]
  msr <- roi_meta$measure[i]

  meanGM_col <- if (msr == "thickness") {
    "Thickness_slope_A0"} else {"Area_slope_A0"}

  df <- data.frame(
```

```
    ROI     = HABS_slopes[[cl]],
    MeanGM  = HABS_slopes[[meanGM_col]],
    gCog    = HABS_slopes$gCog_slope_A0
  )
  df <- df[complete.cases(df), ]
  sm <- summary(lm(gCog ~ ROI + MeanGM, data = df))$coefficients["ROI", ]
  data.frame(
    region  = roi_meta$region[i],
    measure = roi_meta$measure[i],
    beta    = round(sm["Estimate"], 2),
    p       = sm["Pr(>|t|)"],
    n       = nrow(df),
    stringsAsFactors = FALSE
  )
}))

roi_long$p_fdr <- ave(roi_long$p, roi_long$measure, FUN = function(x)
p.adjust(x, method = "fdr"))
roi_table <- reshape(roi_long, idvar = "region",timevar = "measure",
direction = "wide")
p_cols <- grep("^p(\\.|_fdr\\.)", names(roi_table), value = TRUE)
roi_table[p_cols] <- lapply(roi_table[p_cols],function(x) sub("^0\\.", ".",
sprintf("%.3f", x)))
roi_table %>%
  flextable() %>%
  set_caption(
    caption = "ROI Model Results"
  ) %>%
  theme_box() %>%
  fontsize(size = 9, part = "all") %>%
  autofit()
```

ROI Model Results

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
inferiorparietal	0.00	.993	1,252	.996	-0.03	.448	1,252	.662
middletemporal	-0.01	.887	1,252	.962	0.04	.183	1,252	.662
rostralmiddlefrontal	-0.08	.028	1,252	.137	0.03	.300	1,252	.662
bankssts	0.00	.906	1,252	.962	0.01	.653	1,252	.841
caudalanteriorcingulate	-0.01	.860	1,252	.962	0.01	.690	1,252	.841
caudalmiddlefronta	-0.01	.705	1,252	.959	0.04	.189	1,25	.662

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
l							2	
cuneus	0.02	.519	1,252	.841	-0.03	.352	1,252	.662
entorhinal	0.05	.092	1,252	.314	0.03	.291	1,252	.662
frontalpole	-0.03	.358	1,252	.734	0.01	.692	1,252	.841
fusiform	0.07	.027	1,252	.137	-0.05	.074	1,252	.662
inferiortemporal	0.05	.177	1,252	.501	-0.04	.157	1,252	.662
insula	0.08	.015	1,252	.137	0.00	.982	1,252	.982
isthmuscingulate	-0.02	.494	1,252	.840	0.03	.344	1,252	.662
lateraloccipital	-0.03	.474	1,252	.840	-0.03	.357	1,252	.662
lateralorbitofrontal	-0.08	.022	1,252	.137	0.05	.086	1,252	.662
lingual	-0.02	.549	1,252	.848	0.04	.232	1,252	.662
medialorbitofrontal	-0.07	.034	1,252	.147	0.02	.443	1,252	.662
paracentral	0.02	.582	1,252	.861	-0.05	.109	1,252	.662
parahippocampal	0.10	.001	1,252	.029	0.01	.797	1,252	.874
parsopercularis	-0.08	.044	1,252	.166	0.03	.367	1,252	.662
parsorbitalis	-0.03	.337	1,252	.734	-0.01	.772	1,252	.874
parstriangularis	-0.12	.002	1,252	.030	0.06	.108	1,252	.662
pericalcarine	0.00	.877	1,252	.962	0.03	.420	1,252	.662
postcentral	-0.02	.630	1,252	.893	-0.06	.074	1,252	.662

region	beta.thickness	p.thickness	n.thickness	p_fdr.thickness	beta.area	p.area	n.area	p_fdr.area
posteriorcingulate	0.01	.836	1,252	.962	0.02	.471	1,252	.667
precentral	0.04	.358	1,252	.734	-0.03	.341	1,252	.662
precuneus	0.01	.772	1,252	.962	0.04	.334	1,252	.662
rostralanteriorcingulate	0.01	.816	1,252	.962	0.01	.765	1,252	.874
superiorfrontal	0.03	.374	1,252	.734	0.03	.429	1,252	.662
superiorparietal	0.00	.996	1,252	.996	-0.04	.296	1,252	.662
superiortemporal	0.04	.329	1,252	.734	0.00	.943	1,252	.982
supramarginal	-0.07	.148	1,252	.458	0.00	.976	1,252	.982
temporalpole	0.07	.023	1,252	.137	0.01	.605	1,252	.822
transversetemporal	-0.03	.389	1,252	.734	-0.02	.425	1,252	.662

```
HABS_slopes$Thickness_baseline <- scale(HABS_slopes$Thickness_baseline)
HABS_slopes$Area_baseline <- scale(HABS_slopes$Area_baseline)

models_sb <- lapply(adjustments, function(adj) {
  yvar <- paste0("gCog_slope_", adj)
  x_th <- paste0("Thickness_slope_", adj)
  x_ar <- paste0("Area_slope_", adj)
  b_th <- paste0("Thickness_baseline")
  b_ar <- paste0("Area_baseline")
  fml <- as.formula(
    paste(yvar, "~", x_th, "+", x_ar, "+", b_th, "+", b_ar)
  )
  lm(fml, data = HABS_slopes)
})
names(models_sb) <- paste0("m2_", adjustments)

results <- do.call(rbind, lapply(names(models_sb), function(nm) {
  m <- models_sb[[nm]]
  sm <- summary(m)$coefficients
  R2 <- summary(m)$adj.r.squared
  eta <- eta_squared(m, partial = TRUE)
  tmp <- rsq::rsq.partial(m)
```

```
  pr2 <- tibble(predictor = tmp$variable, pr2 = tmp$partial.rsq)
  as.data.frame(sm) %>%
    tibble::rownames_to_column("predictor") %>%
    dplyr::filter(predictor != "(Intercept)") %>%
    dplyr::left_join(eta %>% dplyr::select(Parameter, Eta2_partial),
                     by = c("predictor" = "Parameter")) %>%
    dplyr::left_join(pr2, by = "predictor") %>%
    dplyr::transmute(model = nm,
                     predictor = predictor,
                     beta = Estimate,
                     p = `Pr(>|t|)`,
                     eta2 = Eta2_partial,
                     # R2 = R2,
                     pr2 = pr2)
}))
```

```
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
:
##  extra argument 'family' will be disregarded
## Warning: In lm.fit(x, y, offset = offset, singular.ok = singular.ok, ...)
```

```
:
##  extra argument 'family' will be disregarded

results <- results %>%
dplyr::filter(grepl("Thickness_slope|Area_slope|Thickness_baseline|Area_basel
ine", predictor))

wide_models <- results %>%
  dplyr::mutate(
    Adjustment = dplyr::case_when(
      grepl("^m2_A0$", model)    ~ "A0",
      grepl("^m2_A0dA$", model)  ~ "A0dA",
      grepl("^m2_gamm$", model)  ~ "GAMM",
      TRUE ~ NA_character_
    ),
    Predictor_clean = dplyr::case_when(
      grepl("Thickness_slope_", predictor) ~ "Thickness_slope",
      grepl("Area_slope_", predictor)      ~ "Area_slope",
      grepl("Thickness_baseline", predictor) ~ "Thickness_baseline",
      grepl("Area_baseline", predictor)      ~ "Area_baseline",
      TRUE ~ NA_character_
    ),
    beta_r = round(beta, 2),
    pr2_r  = round(pr2, 3),
    p_r    = round(p, 3),
    stat = paste0(
      beta_r,
      ", R²unique=", pr2_r * 100,
      ", p=", sub("^0\\.", ".", p_r)
    )
  ) %>%
  dplyr::select(Adjustment, Predictor_clean, stat) %>%
  tidyr::pivot_wider(names_from = Predictor_clean, values_from = stat) %>%
  dplyr::arrange(factor(Adjustment, levels = c("A0", "A0dA", "GAMM"))) %>%
  dplyr::select(Adjustment, Thickness_slope, Area_slope, Thickness_baseline,
Area_baseline)

extract_lh <- function(mod, hyp) {
  ct <- as.data.frame(car::linearHypothesis(mod, hyp))
  row <- ct[2, , drop = FALSE]
  stat_col <- intersect(c("Chisq", "F"), names(row))[1]
  p_col <- intersect(c("Pr(>Chisq)", "Pr(>F)"), names(row))[1]
  stat_val <- suppressWarnings(as.numeric(row[[stat_col]]))
  p_val <- suppressWarnings(as.numeric(row[[p_col]]))
  tibble(stat_type = stat_col,
         stat = round(stat_val, 2),
         p_lh = sub("^0\\.", ".", sprintf("%.3f", p_val)))
}

lh_table <- tibble(
```

```
  Adjustment = c("A0", "A0dA", "GAMM"),
  model_name = c("m2_A0", "m2_A0dA", "m2_gamm"),
  contrast = c(
    "Thickness_slope_A0 - Area_slope_A0 = 0",
    "Thickness_slope_A0dA - Area_slope_A0dA = 0",
    "Thickness_slope_gamm - Area_slope_gamm = 0"
  )
) %>%
  dplyr::mutate(out = purrr::pmap(list(model_name, contrast),\(mn, h)
extract_lh(models_sb[[mn]], h))) %>%
  tidyr::unnest(out) %>%
  dplyr::select(-model_name, -contrast)

final_models_table <- wide_models %>% dplyr::left_join(lh_table, by =
"Adjustment")
final_models_table %>% flextable() %>% set_caption(caption = "Slope +
Baseline Results")
```

Slope + Baseline Results

Adjustment	Thickness_slope	Area_slope	Thickness_baseline	Area_baseline	stat_type	stat	p_lh
A0	0.12, R²uniqu e=1.3, p=0	0.05, R²uniqu e=0.3, p=.061	0.12, R²uniqu e=1.4, p=0	0, R²uniqu e=0, p=.889	F	2.21	.137
A0dA	0.12, R²uniqu e=1.4, p=0	0.04, R²uniqu e=0.2, p=.12	0.12, R²uniqu e=1.2, p=0	0, R²uniqu e=0, p=.946	F	3.21	.073
GAMM	0.12, R²uniqu e=1.3, p=0	0.04, R²uniqu e=0.2, p=.151	0.11, R²uniqu e=1.1, p=0	0, R²uniqu e=0, p=.973	F	3.29	.070

```
summary(lm(gCog_slope_gamm ~ Thickness_slope_gamm + Thickness_baseline +
Area_slope_gamm + Area_baseline, data = HABS_slopes))

##
## Call:
## lm(formula = gCog_slope_gamm ~ Thickness_slope_gamm + Thickness_baseline +
##     Area_slope_gamm + Area_baseline, data = HABS_slopes)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -3.4786 -0.5481  0.0176  0.5773  3.1014
##
```

```
## Coefficients:
##                          Estimate Std. Error t value Pr(>|t|)
## (Intercept)          -3.233e-15  2.798e-02   0.000 1.000000
## Thickness_slope_gamm  1.214e-01  2.959e-02   4.105 4.31e-05 ***
## Thickness_baseline    1.084e-01  2.924e-02   3.708 0.000218 ***
## Area_slope_gamm       4.104e-02  2.859e-02   1.436 0.151364
## Area_baseline        -9.482e-04  2.842e-02  -0.033 0.973388
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.9902 on 1247 degrees of freedom
##   (63 observations deleted due to missingness)
## Multiple R-squared:  0.02271,    Adjusted R-squared:  0.01958
## F-statistic: 7.246 on 4 and 1247 DF,  p-value: 9.101e-06
```

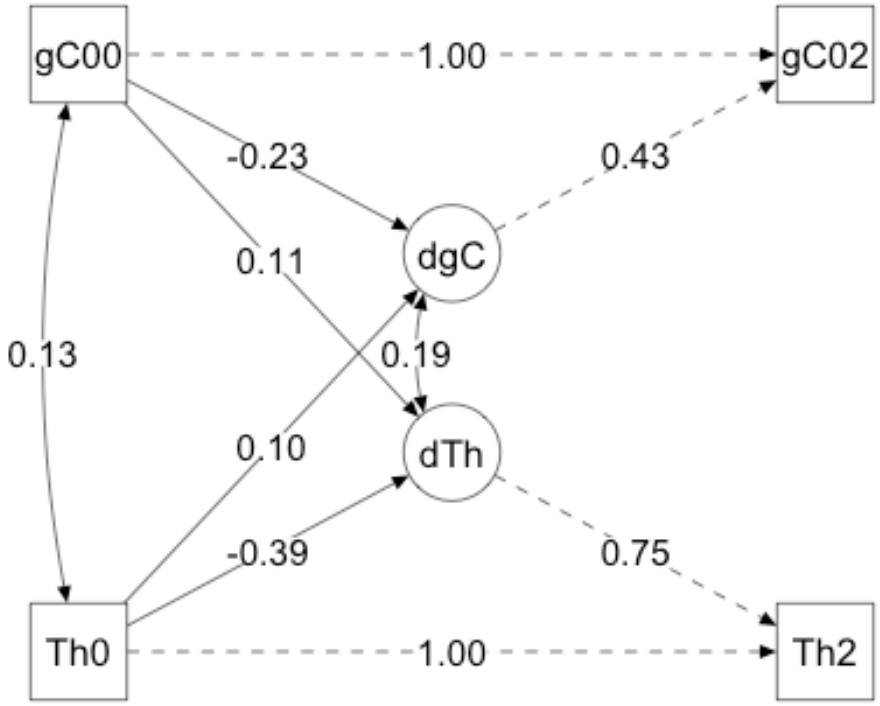
```
mydata_growth <- mydata_slopes[!is.na(mydata_slopes$Thickness0) &
!is.na(mydata_slopes$Thickness1) &
                               !is.na(mydata_slopes$Thickness2) &
                               !is.na(mydata_slopes$Area0) &
!is.na(mydata_slopes$Area1) &
                               !is.na(mydata_slopes$Area2), ]

cols_to_scale <- c("Thickness0", "Thickness1", "Thickness2", "Area0",
"Area1", "Area2",
                   "gCog00", "gCog01", "gCog02")

mydata_growth[cols_to_scale] <- scale(mydata_growth[cols_to_scale])

BLCS_thickness <-'
Thickness2 ~ 1*Thickness0
dThickness =~ 1*Thickness2
Thickness2 ~ 0*1
Thickness2 ~~ 0*Thickness2

gCog02 ~ 1*gCog00
dgCog =~ 1*gCog02
dgCog ~ 1
gCog00 ~  1
gCog02 ~ 0*1

dgCog ~~  dgCog
gCog00 ~~   gCog00
gCog02 ~~ 0*gCog02

dThickness ~ 1
Thickness0 ~ 1
dThickness ~~ dThickness
Thickness0 ~~ Thickness0
```

```
dThickness~gCog00+Thickness0
dgCog~Thickness0+gCog00

gCog00 ~~  Thickness0
dgCog ~~ dThickness
'
fitBLCS_thickness <- lavaan(BLCS_thickness, data=mydata_growth,
estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_thickness, fit.measures=TRUE, standardized=TRUE,
rsquare=TRUE)

## lavaan 0.6-21 ended normally after 31 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        14
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##   Test Statistic                                 0.000       0.000
##   Degrees of freedom                                 0           0
##
## Model Test Baseline Model:
##
##   Test statistic                              1403.846    1145.802
##   Degrees of freedom                                 6           6
##   P-value                                        0.000       0.000
##   Scaling correction factor                                  1.225
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    1.000       1.000
##   Tucker-Lewis Index (TLI)                       1.000       1.000
##
##   Robust Comparative Fit Index (CFI)                         1.000
##   Robust Tucker-Lewis Index (TLI)                            1.000
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -2434.767   -2434.767
##   Loglikelihood unrestricted model (H1)      -2434.767   -2434.767
##
##   Akaike (AIC)                                4897.534    4897.534
##   Bayesian (BIC)                              4957.949    4957.949
##   Sample-size adjusted Bayesian (SABIC)       4913.507    4913.507
##
## Root Mean Square Error of Approximation:
```

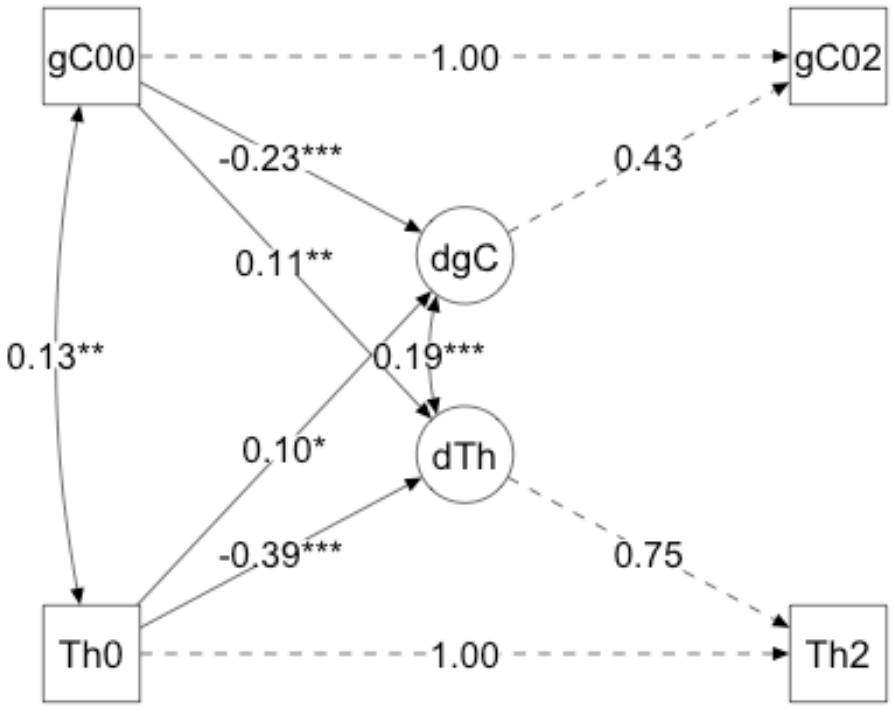
```
##
##   RMSEA                                          0.000          NA
##   90 Percent confidence interval - lower         0.000          NA
##   90 Percent confidence interval - upper         0.000          NA
##   P-value H_0: RMSEA <= 0.050                       NA          NA
##   P-value H_0: RMSEA >= 0.080                       NA          NA
##
##   Robust RMSEA                                                0.000
##   90 Percent confidence interval - lower                      0.000
##   90 Percent confidence interval - upper                      0.000
##   P-value H_0: Robust RMSEA <= 0.050                             NA
##   P-value H_0: Robust RMSEA >= 0.080                             NA
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.000       0.000
##
## Parameter Estimates:
##
##   Standard errors                             Sandwich
##   Information bread                           Observed
##   Observed information based on                Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dThickness =~
##     Thickness2        1.000                               0.747    0.747
##   dgCog =~
##     gCog02            1.000                               0.432    0.432
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Thickness2 ~
##     Thickness0        1.000                               1.000    1.000
##   gCog02 ~
##     gCog00            1.000                               1.000    1.000
##   dThickness ~
##     gCog00            0.078    0.029    2.691    0.007    0.105    0.105
##     Thickness0       -0.290    0.033   -8.732    0.000   -0.388   -0.388
##   dgCog ~
##     Thickness0        0.042    0.019    2.206    0.027    0.097    0.097
##     gCog00           -0.099    0.019   -5.341    0.000   -0.229   -0.229
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Thickness0 ~~
##     gCog00            0.132    0.042    3.119    0.002    0.132    0.133
##  .dThickness ~~
##    .dgCog             0.056    0.013    4.177    0.000    0.194    0.194
##
```

```
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Thickness2        0.000                               0.000    0.000
##    .dgCog            -0.000    0.018   -0.000    1.000   -0.000   -0.000
##     gCog00           -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .gCog02            0.000                               0.000    0.000
##    .dThickness        0.000    0.029    0.000    1.000    0.000    0.000
##     Thickness0       -0.000    0.042   -0.000    1.000   -0.000   -0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Thickness2        0.000                               0.000    0.000
##    .dgCog             0.176    0.012   14.421    0.000    0.944    0.944
##     gCog00            0.998    0.056   17.791    0.000    0.998    1.000
##    .gCog02            0.000                               0.000    0.000
##    .dThickness        0.474    0.033   14.212    0.000    0.850    0.850
##     Thickness0        0.998    0.056   17.889    0.000    0.998    1.000
##
## R-Square:
##                    Estimate
##     Thickness2        1.000
##     dgCog             0.056
##     gCog02            1.000
##     dThickness        0.150
```

```r
thickness_plot <- semPlot::semPaths(fitBLCS_thickness,
                  whatLabels = "std",
                  layout = "tree",
                  rotation = 2,
                  edge.label.cex = 1.5,
                  label.cex = 1.2,
                  sizeMan = 9,
                  sizeLat = 9,
                  nCharNodes = 3,
                  intercepts = FALSE,
                  residuals = FALSE,
                  edge.color = "black")
```




```
thickness_plot <- mark_sig(thickness_plot, fitBLCS_thickness)
plot(thickness_plot)
```




```
BLCS_thickness_A0 <-'
Thickness2 ~ 1*Thickness0
dThickness =~ 1*Thickness2
Thickness2 ~ 0*1
Thickness2 ~~ 0*Thickness2

gCog02 ~ 1*gCog00
dgCog =~ 1*gCog02
dgCog ~ 1
gCog00 ~  1
gCog02 ~ 0*1

dgCog ~~  dgCog
gCog00 ~~   gCog00
gCog02 ~~ 0*gCog02

dThickness ~ 1
Thickness0 ~ 1
dThickness ~~ dThickness
Thickness0 ~~ Thickness0

dThickness~gCog00+Thickness0
```

```
dgCog~Thickness0+gCog00

gCog00 ~~  Thickness0
dgCog ~~ dThickness

dThickness ~ A0
dgCog      ~ A0
'
fitBLCS_thickness_A0 <- lavaan(BLCS_thickness_A0, data=mydata_growth,
estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_thickness_A0, fit.measures=TRUE, standardized=TRUE,
rsquare=TRUE)

## lavaan 0.6-21 ended normally after 47 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        16
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##                                              Standard      Scaled
##   Test Statistic                               52.196      52.866
##   Degrees of freedom                                2           2
##   P-value (Chi-square)                          0.000       0.000
##   Scaling correction factor                                 0.987
##     Yuan-Bentler correction (Mplus variant)
##
## Model Test Baseline Model:
##
##   Test statistic                             1482.264    1324.621
##   Degrees of freedom                               10          10
##   P-value                                       0.000       0.000
##   Scaling correction factor                                 1.119
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                   0.966       0.961
##   Tucker-Lewis Index (TLI)                      0.830       0.807
##
##   Robust Comparative Fit Index (CFI)                        0.966
##   Robust Tucker-Lewis Index (TLI)                           0.830
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -2421.656   -2421.656
##   Scaling correction factor                                 1.048
##       for the MLR correction
```

```
##   Loglikelihood unrestricted model (H1)      -2395.558   -2395.558
##   Scaling correction factor                                  1.041
##       for the MLR correction
##
##   Akaike (AIC)                                4875.313    4875.313
##   Bayesian (BIC)                              4944.358    4944.358
##   Sample-size adjusted Bayesian (SABIC)       4893.567    4893.567
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.213       0.214
##   90 Percent confidence interval - lower         0.165       0.167
##   90 Percent confidence interval - upper         0.265       0.267
##   P-value H_0: RMSEA <= 0.050                    0.000       0.000
##   P-value H_0: RMSEA >= 0.080                    1.000       1.000
##
##   Robust RMSEA                                               0.213
##   90 Percent confidence interval - lower                     0.165
##   90 Percent confidence interval - upper                     0.265
##   P-value H_0: Robust RMSEA <= 0.050                         0.000
##   P-value H_0: Robust RMSEA >= 0.080                         1.000
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.089       0.089
##
## Parameter Estimates:
##
##   Standard errors                             Sandwich
##   Information bread                           Observed
##   Observed information based on                Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dThickness =~
##     Thickness2        1.000                               0.758    0.775
##   dgCog =~
##     gCog02            1.000                               0.433    0.438
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Thickness2 ~
##     Thickness0        1.000                               1.000    1.022
##   gCog02 ~
##     gCog00            1.000                               1.000    1.010
##   dThickness ~
##     gCog00            0.069    0.029    2.410    0.016    0.091    0.091
##     Thickness0       -0.318    0.034   -9.456    0.000   -0.420   -0.420
##   dgCog ~
##     Thickness0        0.019    0.020    0.972    0.331    0.045    0.045
```

```
##     gCog00           -0.106    0.018   -5.885    0.000   -0.245   -0.245
##   dThickness ~
##     A0               -0.013    0.004   -3.541    0.000   -0.017   -0.137
##   dgCog ~
##     A0               -0.010    0.002   -4.570    0.000   -0.024   -0.188
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Thickness0 ~~
##     gCog00            0.132    0.042    3.119    0.002    0.132    0.133
##  .dThickness ~~
##    .dgCog             0.048    0.013    3.779    0.000    0.173    0.173
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Thickness2        0.000                                0.000    0.000
##    .dgCog             0.673    0.144    4.655    0.000    1.554    1.554
##     gCog00           -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .gCog02            0.000                                0.000    0.000
##    .dThickness        0.861    0.246    3.498    0.000    1.136    1.136
##     Thickness0       -0.000    0.042   -0.000    1.000   -0.000   -0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Thickness2        0.000                                0.000    0.000
##    .dgCog             0.170    0.012   14.333    0.000    0.906    0.906
##     gCog00            0.998    0.056   17.791    0.000    0.998    1.000
##    .gCog02            0.000                                0.000    0.000
##    .dThickness        0.464    0.033   14.241    0.000    0.807    0.807
##     Thickness0        0.998    0.056   17.889    0.000    0.998    1.000
##
## R-Square:
##                    Estimate
##     Thickness2        1.000
##     dgCog             0.094
##     gCog02            1.000
##     dThickness        0.193
```

```
BLCS_area <-'
Area2 ~ 1*Area0
dArea =~ 1*Area2
Area2 ~ 0*1
Area2 ~~ 0*Area2

gCog02 ~ 1*gCog00
dgCog =~ 1*gCog02
dgCog ~ 1
gCog00 ~  1
gCog02 ~ 0*1
```

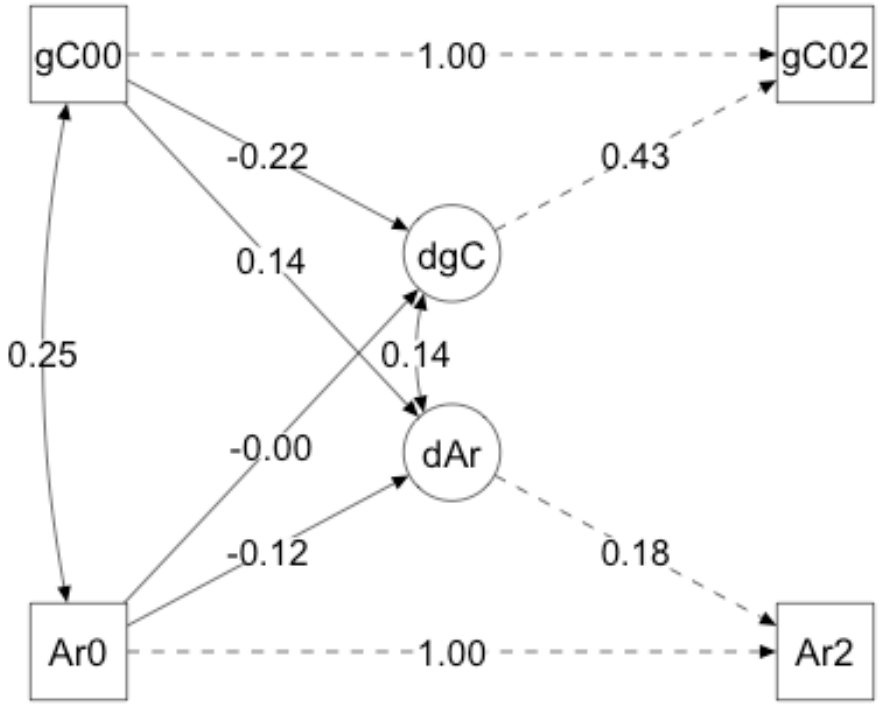
```
dgCog ~~  dgCog
gCog00 ~~   gCog00
gCog02 ~~ 0*gCog02

dArea ~ 1
Area0 ~ 1
dArea ~~ dArea
Area0 ~~ Area0

dArea~gCog00+Area0
dgCog~Area0+gCog00

gCog00 ~~  Area0
dgCog ~~ dArea
'
fitBLCS_area <- lavaan(BLCS_area, data=mydata_growth,
estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_area, fit.measures=TRUE, standardized=TRUE, rsquare=TRUE)
```

```
## lavaan 0.6-21 ended normally after 30 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        14
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##   Test Statistic                                 0.000       0.000
##   Degrees of freedom                                 0           0
##
## Model Test Baseline Model:
##
##   Test statistic                              2930.649    2065.410
##   Degrees of freedom                                 6           6
##   P-value                                        0.000       0.000
##   Scaling correction factor                                  1.419
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    1.000       1.000
##   Tucker-Lewis Index (TLI)                       1.000       1.000
##
##   Robust Comparative Fit Index (CFI)                         1.000
##   Robust Tucker-Lewis Index (TLI)                            1.000
##
## Loglikelihood and Information Criteria:
##
```

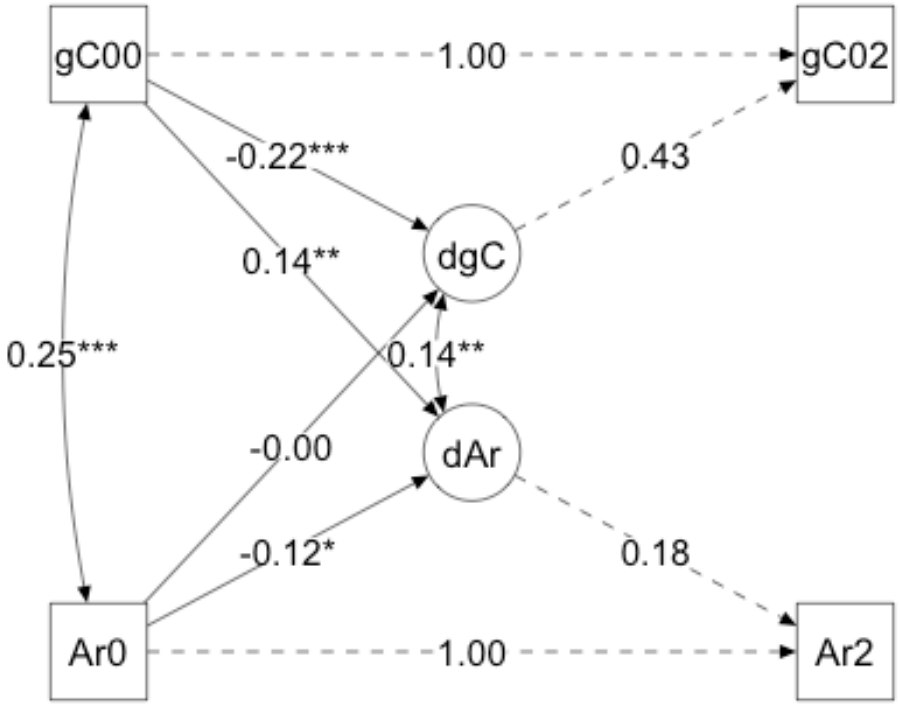
```
##   Loglikelihood user model (H0)              -1671.366   -1671.366
##   Loglikelihood unrestricted model (H1)      -1671.366   -1671.366
##
##   Akaike (AIC)                                3370.732    3370.732
##   Bayesian (BIC)                              3431.147    3431.147
##   Sample-size adjusted Bayesian (SABIC)       3386.705    3386.705
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.000          NA
##   90 Percent confidence interval - lower         0.000          NA
##   90 Percent confidence interval - upper         0.000          NA
##   P-value H_0: RMSEA <= 0.050                       NA          NA
##   P-value H_0: RMSEA >= 0.080                       NA          NA
##
##   Robust RMSEA                                               0.000
##   90 Percent confidence interval - lower                     0.000
##   90 Percent confidence interval - upper                     0.000
##   P-value H_0: Robust RMSEA <= 0.050                            NA
##   P-value H_0: Robust RMSEA >= 0.080                            NA
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.000       0.000
##
## Parameter Estimates:
##
##   Standard errors                             Sandwich
##   Information bread                           Observed
##   Observed information based on                Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dArea =~
##     Area2             1.000                                0.177    0.177
##   dgCog =~
##     gCog02            1.000                                0.432    0.432
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Area2 ~
##     Area0             1.000                                1.000    1.000
##   gCog02 ~
##     gCog00            1.000                                1.000    1.000
##   dArea ~
##     gCog00            0.024    0.008    3.104    0.002    0.136    0.136
##     Area0            -0.022    0.009   -2.382    0.017   -0.123   -0.123
##   dgCog ~
##     Area0            -0.001    0.019   -0.040    0.968   -0.002   -0.002
##     gCog00           -0.093    0.018   -5.040    0.000   -0.216   -0.216
```

```
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Area0 ~~
##     gCog00            0.251    0.038    6.656    0.000    0.251    0.252
##  .dArea ~~
##    .dgCog             0.010    0.003    3.071    0.002    0.138    0.138
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Area2             0.000                               0.000    0.000
##    .dgCog             0.000    0.018    0.000    1.000    0.000    0.000
##     gCog00           -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .gCog02            0.000                               0.000    0.000
##    .dArea             0.000    0.007    0.000    1.000    0.000    0.000
##     Area0             0.000    0.042    0.000    1.000    0.000    0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Area2             0.000                               0.000    0.000
##    .dgCog             0.178    0.012   14.236    0.000    0.953    0.953
##     gCog00            0.998    0.056   17.791    0.000    0.998    1.000
##    .gCog02            0.000                               0.000    0.000
##    .dArea             0.031    0.003   10.832    0.000    0.975    0.975
##     Area0             0.998    0.058   17.318    0.000    0.998    1.000
##
## R-Square:
##                    Estimate
##     Area2             1.000
##     dgCog             0.047
##     gCog02            1.000
##     dArea             0.025
```

```r
area_plot <- semPlot::semPaths(fitBLCS_area,
                  whatLabels = "std",
                  layout = "tree",
                  rotation = 2,
                  edge.label.cex = 1.5,
                  label.cex = 1.2,
                  sizeMan = 9,
                  sizeLat = 9,
                  nCharNodes = 3,
                  intercepts = FALSE,
                  residuals = FALSE,
                  edge.color = "black")
```




```
area_plot <- mark_sig(area_plot, fitBLCS_area)
plot(area_plot)
```




```
BLCS_area_A0 <-'
Area2 ~ 1*Area0
dArea =~ 1*Area2
Area2 ~ 0*1
Area2 ~~ 0*Area2

gCog02 ~ 1*gCog00
dgCog =~ 1*gCog02
dgCog ~ 1
gCog00 ~  1
gCog02 ~ 0*1

dgCog ~~  dgCog
gCog00 ~~   gCog00
gCog02 ~~ 0*gCog02

dArea ~ 1
Area0 ~ 1
dArea ~~ dArea
Area0 ~~ Area0

dArea~gCog00+Area0
```

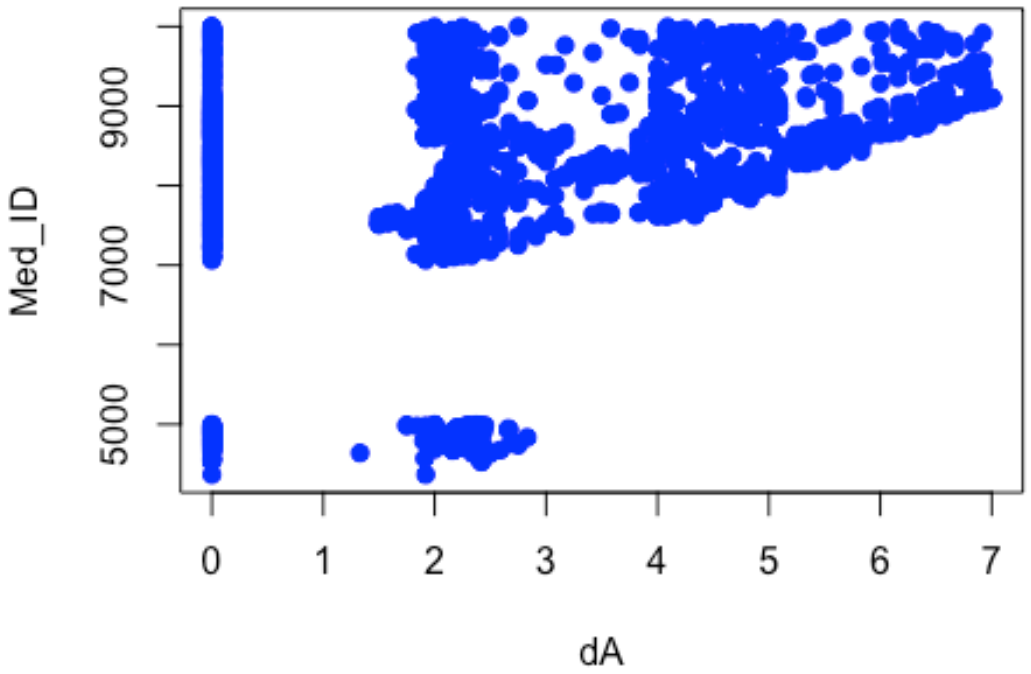
```
dgCog~Area0+gCog00

gCog00 ~~  Area0
dgCog ~~ dArea

dArea ~ A0
dgCog ~ A0
'
fitBLCS_area_A0 <- lavaan(BLCS_area_A0, data=mydata_growth,
estimator='mlr',fixed.x=TRUE,missing='fiml')
summary(fitBLCS_area_A0, fit.measures=TRUE, standardized=TRUE, rsquare=TRUE)
```

```
## lavaan 0.6-21 ended normally after 48 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        16
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##                                               Standard      Scaled
##   Test Statistic                                11.427      11.359
##   Degrees of freedom                                 2           2
##   P-value (Chi-square)                           0.003       0.003
##   Scaling correction factor                                  1.006
##     Yuan-Bentler correction (Mplus variant)
##
## Model Test Baseline Model:
##
##   Test statistic                              2971.176    2404.220
##   Degrees of freedom                                10          10
##   P-value                                        0.000       0.000
##   Scaling correction factor                                  1.236
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.997       0.996
##   Tucker-Lewis Index (TLI)                       0.984       0.980
##
##   Robust Comparative Fit Index (CFI)                         0.997
##   Robust Tucker-Lewis Index (TLI)                            0.984
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -1656.816   -1656.816
##   Scaling correction factor                                  1.125
##       for the MLR correction
##   Loglikelihood unrestricted model (H1)      -1651.102   -1651.102
```

```
##   Scaling correction factor                                  1.112
##       for the MLR correction
##
##   Akaike (AIC)                                       3345.631    3345.631
##   Bayesian (BIC)                                     3414.677    3414.677
##   Sample-size adjusted Bayesian (SABIC)              3363.886    3363.886
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                                 0.092       0.092
##   90 Percent confidence interval - lower                0.046       0.045
##   90 Percent confidence interval - upper                0.147       0.147
##   P-value H_0: RMSEA <= 0.050                           0.066       0.067
##   P-value H_0: RMSEA >= 0.080                           0.709       0.706
##
##   Robust RMSEA                                                      0.092
##   90 Percent confidence interval - lower                            0.045
##   90 Percent confidence interval - upper                            0.148
##   P-value H_0: Robust RMSEA <= 0.050                                0.068
##   P-value H_0: Robust RMSEA >= 0.080                                0.706
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                                  0.049       0.049
##
## Parameter Estimates:
##
##   Standard errors                                    Sandwich
##   Information bread                                  Observed
##   Observed information based on                       Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   dArea =~
##     Area2             1.000                               0.177    0.177
##   dgCog =~
##     gCog02            1.000                               0.434    0.439
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Area2 ~
##     Area0             1.000                               1.000    1.002
##   gCog02 ~
##     gCog00            1.000                               1.000    1.010
##   dArea ~
##     gCog00            0.022    0.008    2.830    0.005    0.123    0.123
##     Area0            -0.023    0.009   -2.567    0.010   -0.132   -0.132
##   dgCog ~
##     Area0            -0.007    0.019   -0.371    0.711   -0.016   -0.016
##     gCog00           -0.102    0.018   -5.694    0.000   -0.236   -0.235
```

```
##   dArea ~
##     A0               -0.003    0.001   -3.108    0.002   -0.016   -0.124
##   dgCog ~
##     A0               -0.011    0.002   -5.140    0.000   -0.025   -0.200
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   Area0 ~~
##     gCog00            0.251    0.038    6.656    0.000    0.251    0.252
##  .dArea ~~
##    .dgCog             0.008    0.003    2.638    0.008    0.116    0.116
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Area2             0.000                              0.000    0.000
##    .dgCog             0.721    0.138    5.225    0.000    1.660    1.660
##     gCog00           -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .gCog02            0.000                              0.000    0.000
##    .dArea             0.182    0.058    3.121    0.002    1.029    1.029
##     Area0             0.000    0.042    0.000    1.000    0.000    0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .Area2             0.000                              0.000    0.000
##    .dgCog             0.170    0.012   14.262    0.000    0.902    0.902
##     gCog00            0.998    0.056   17.791    0.000    0.998    1.000
##    .gCog02            0.000                              0.000    0.000
##    .dArea             0.030    0.003   10.605    0.000    0.960    0.960
##     Area0             0.998    0.058   17.318    0.000    0.998    1.000
##
## R-Square:
##                    Estimate
##     Area2             1.000
##     dgCog             0.098
##     gCog02            1.000
##     dArea             0.040
```

```r
# bad_ids <- mydata_long$Med_ID[mydata_long$dA >= 2.5 & mydata_long$dA <= 3.5
|
#                               mydata_long$dA >= 5 & mydata_long$dA <= 6]
#
# mydata_filtered <- subset(mydata_growth, !(Med_ID %in% bad_ids))

plot(mydata_long$dA, mydata_long$Med_ID,
     xlab = "dA", ylab = "Med_ID",
     pch = 19, col = "blue")
```




```
lgm_model_thick <- '

  i_gCog =~ 1*gCog00 + 1*gCog01 + 1*gCog02
  s_gCog =~ 0*gCog00 + 1*gCog01 + 2*gCog02

  i_Thick =~ 1*Thickness0 + 1*Thickness1 + 1*Thickness2
  s_Thick =~ 0*Thickness0 + 1*Thickness1 + 2*Thickness2

  i_gCog ~ 1
  s_gCog ~ 1

  i_Thick ~ 1
  s_Thick ~ 1

  i_gCog ~~ i_Thick
  s_gCog ~~ s_Thick
  s_Thick ~ i_gCog
  s_gCog ~ i_Thick

  gCog00 ~~ 0.01*gCog00
  gCog01 ~~ 0.01*gCog01
  gCog02 ~~ 0.01*gCog02
```

```
  Thickness0 ~~ 0.01*Thickness0
  Thickness1 ~~ 0.01*Thickness1
  Thickness2 ~~ 0.01*Thickness2

'

fit_lgm_thick <- growth(lgm_model_thick, data = mydata_growth, missing =
"fiml")
summary(fit_lgm_thick, fit.measures = TRUE, standardized = TRUE, rsquare =
TRUE)

## lavaan 0.6-21 ended normally after 28 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        12
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##
##   Test statistic                             11105.206
##   Degrees of freedom                                15
##   P-value (Chi-square)                           0.000
##
## Model Test Baseline Model:
##
##   Test statistic                              3351.838
##   Degrees of freedom                                15
##   P-value                                        0.000
##
## User Model versus Baseline Model:
##
##   Comparative Fit Index (CFI)                    0.000
##   Tucker-Lewis Index (TLI)                      -2.324
##
##   Robust Comparative Fit Index (CFI)             0.000
##   Robust Tucker-Lewis Index (TLI)               -2.324
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -8581.719
##   Loglikelihood unrestricted model (H1)      -3029.116
##
##   Akaike (AIC)                               17187.438
##   Bayesian (BIC)                             17239.223
##   Sample-size adjusted Bayesian (SABIC)      17201.129
##
```

```
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          1.156
##   90 Percent confidence interval - lower         1.138
##   90 Percent confidence interval - upper         1.174
##   P-value H_0: RMSEA <= 0.050                    0.000
##   P-value H_0: RMSEA >= 0.080                    1.000
##
##   Robust RMSEA                                   1.156
##   90 Percent confidence interval - lower         1.138
##   90 Percent confidence interval - upper         1.174
##   P-value H_0: Robust RMSEA <= 0.050             0.000
##   P-value H_0: Robust RMSEA >= 0.080             1.000
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.156
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Observed
##   Observed information based on                Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   i_gCog =~
##     gCog00            1.000                               0.987    0.995
##     gCog01            1.000                               0.987    0.971
##     gCog02            1.000                               0.987    0.913
##   s_gCog =~
##     gCog00            0.000                               0.000    0.000
##     gCog01            1.000                               0.205    0.201
##     gCog02            2.000                               0.410    0.379
##   i_Thick =~
##     Thickness0        1.000                               0.975    0.995
##     Thickness1        1.000                               0.975    0.928
##     Thickness2        1.000                               0.975    0.792
##   s_Thick =~
##     Thickness0        0.000                               0.000    0.000
##     Thickness1        1.000                               0.366    0.348
##     Thickness2        2.000                               0.732    0.594
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   s_Thick ~
##     i_gCog            0.036    0.016    2.259    0.024    0.098    0.098
##   s_gCog ~
##     i_Thick           0.028    0.010    2.795    0.005    0.132    0.132
##
```

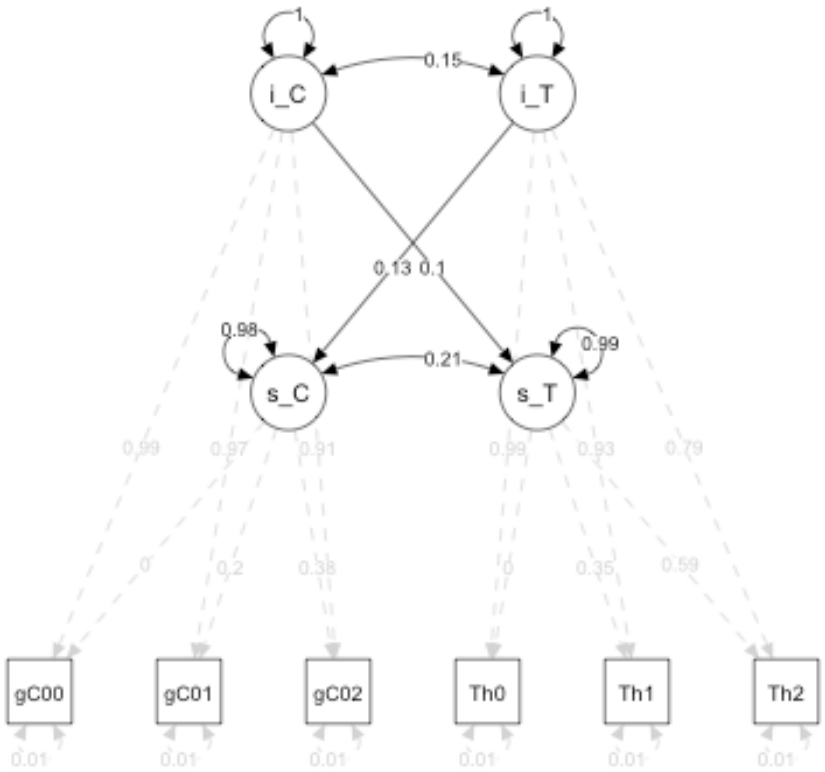
```
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   i_gCog ~~
##     i_Thick           0.142    0.042    3.397    0.001    0.147    0.147
##  .s_gCog ~~
##    .s_Thick           0.015    0.004    4.073    0.000    0.208    0.208
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##     i_gCog            0.000    0.042    0.000    1.000    0.000    0.000
##    .s_gCog            0.000    0.009    0.000    1.000    0.000    0.000
##     i_Thick          -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .s_Thick          -0.000    0.016   -0.000    1.000   -0.000   -0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .gCog00            0.010                               0.010    0.010
##    .gCog01            0.010                               0.010    0.010
##    .gCog02            0.010                               0.010    0.009
##    .Thickness0        0.010                               0.010    0.010
##    .Thickness1        0.010                               0.010    0.009
##    .Thickness2        0.010                               0.010    0.007
##     i_gCog            0.974    0.059   16.495    0.000    1.000    1.000
##    .s_gCog            0.041    0.003   14.749    0.000    0.982    0.982
##     i_Thick           0.951    0.058   16.486    0.000    1.000    1.000
##    .s_Thick           0.133    0.008   15.997    0.000    0.990    0.990
##
## R-Square:
##                    Estimate
##     gCog00            0.990
##     gCog01            0.990
##     gCog02            0.991
##     Thickness0        0.990
##     Thickness1        0.991
##     Thickness2        0.993
##     s_gCog            0.018
##     s_Thick           0.010
```

```
colMeans(is.na(mydata_growth[, c("Thickness0", "Thickness1", "Thickness2")]))
```

```
## Thickness0 Thickness1 Thickness2
##          0          0          0
```

```
param_est <- parameterEstimates(fit_lgm_thick, standardized=TRUE)
param_edges <- param_est[param_est$op %in% c("=~", "~", "~~"), ]

param_edges$pvalue[is.na(param_edges$pvalue)] <- 1
edge_colors <- ifelse(param_edges$pvalue < 0.05, "black", "lightgrey")

edge_labels <- round(param_edges$std.all, 2)
```

```r
semPaths(fit_lgm_thick,
         whatLabels = "none",
         edgeLabels = as.character(edge_labels),
         edge.color = edge_colors,
         layout = "tree",
         rotation = 1,
         edge.label.cex = 0.8,
         edge.label.position = 0.6,
         sizeMan = 6,
         sizeLat = 7,
         esize = 1,
         curve = 1,
         intercepts = FALSE,
         residuals = TRUE
)
```



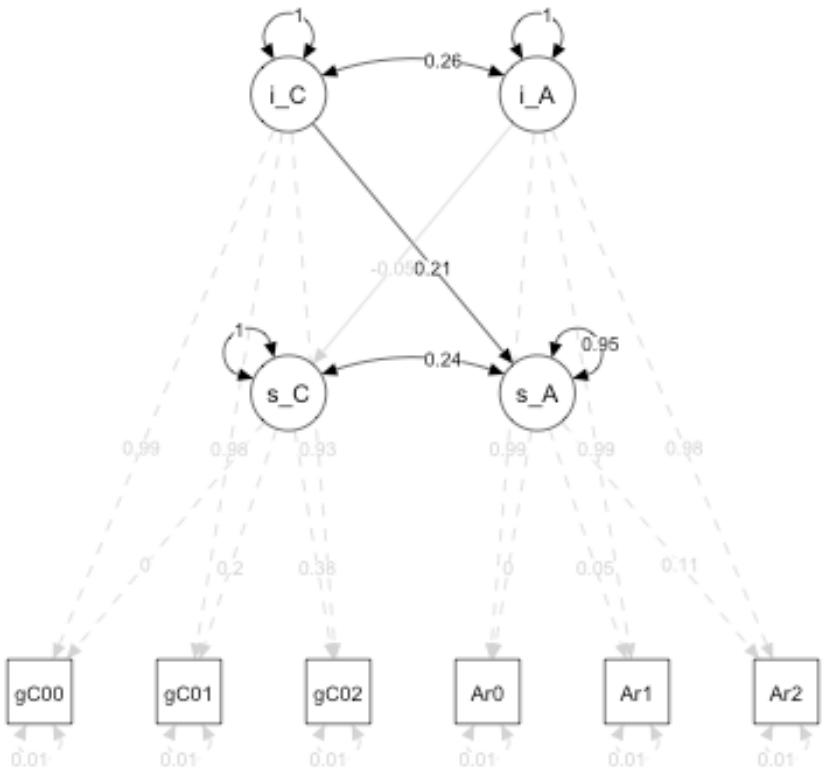

```r
lgm_model_area <- '

  i_gCog =~ 1*gCog00 + 1*gCog01 + 1*gCog02
  s_gCog =~ 0*gCog00 + 1*gCog01 + 2*gCog02

  i_Area =~ 1*Area0 + 1*Area1 + 1*Area2
  s_Area =~ 0*Area0 + 1*Area1 + 2*Area2
```

```
  i_gCog ~ 1
  s_gCog ~ 1

  i_Area ~ 1
  s_Area ~ 1

  i_gCog ~~ i_Area
  s_gCog ~~ s_Area
  s_Area ~ i_gCog
  s_gCog ~ i_Area

  gCog00 ~~ 0.01*gCog00
  gCog01 ~~ 0.01*gCog01
  gCog02 ~~ 0.01*gCog02

  Area0 ~~ 0.01*Area0
  Area1 ~~ 0.01*Area1
  Area2 ~~ 0.01*Area2

'

fit_lgm_area <- growth(lgm_model_area, data = mydata_growth, missing =
"fiml")
summary(fit_lgm_area, fit.measures = TRUE, standardized = TRUE, rsquare =
TRUE)

## lavaan 0.6-21 ended normally after 42 iterations
##
##   Estimator                                         ML
##   Optimization method                           NLMINB
##   Number of model parameters                        12
##
##   Number of observations                           553
##   Number of missing patterns                         1
##
## Model Test User Model:
##
##   Test statistic                              2261.532
##   Degrees of freedom                                15
##   P-value (Chi-square)                           0.000
##
## Model Test Baseline Model:
##
##   Test statistic                              6471.308
##   Degrees of freedom                                15
##   P-value                                        0.000
##
## User Model versus Baseline Model:
```

```
##
##   Comparative Fit Index (CFI)                    0.652
##   Tucker-Lewis Index (TLI)                       0.652
##
##   Robust Comparative Fit Index (CFI)             0.652
##   Robust Tucker-Lewis Index (TLI)                0.652
##
## Loglikelihood and Information Criteria:
##
##   Loglikelihood user model (H0)              -2600.147
##   Loglikelihood unrestricted model (H1)      -1469.381
##
##   Akaike (AIC)                                5224.295
##   Bayesian (BIC)                              5276.079
##   Sample-size adjusted Bayesian (SABIC)       5237.986
##
## Root Mean Square Error of Approximation:
##
##   RMSEA                                          0.520
##   90 Percent confidence interval - lower         0.502
##   90 Percent confidence interval - upper         0.539
##   P-value H_0: RMSEA <= 0.050                    0.000
##   P-value H_0: RMSEA >= 0.080                    1.000
##
##   Robust RMSEA                                   0.520
##   90 Percent confidence interval - lower         0.502
##   90 Percent confidence interval - upper         0.539
##   P-value H_0: Robust RMSEA <= 0.050             0.000
##   P-value H_0: Robust RMSEA >= 0.080             1.000
##
## Standardized Root Mean Square Residual:
##
##   SRMR                                           0.041
##
## Parameter Estimates:
##
##   Standard errors                             Standard
##   Information                                 Observed
##   Observed information based on                Hessian
##
## Latent Variables:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   i_gCog =~
##     gCog00            1.000                               0.988    0.995
##     gCog01            1.000                               0.988    0.977
##     gCog02            1.000                               0.988    0.926
##   s_gCog =~
##     gCog00            0.000                               0.000    0.000
##     gCog01            1.000                               0.203    0.201
##     gCog02            2.000                               0.405    0.380
```

```
##   i_Area =~
##     Area0             1.000                                    0.991    0.995
##     Area1             1.000                                    0.991    0.991
##     Area2             1.000                                    0.991    0.984
##   s_Area =~
##     Area0             0.000                                    0.000    0.000
##     Area1             1.000                                    0.053    0.053
##     Area2             2.000                                    0.106    0.105
##
## Regressions:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   s_Area ~
##     i_gCog            0.011    0.004    3.015    0.003    0.214    0.214
##   s_gCog ~
##     i_Area           -0.011    0.009   -1.195    0.232   -0.053   -0.053
##
## Covariances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##   i_gCog ~~
##     i_Area            0.256    0.043    5.906    0.000    0.262    0.262
##  .s_gCog ~~
##    .s_Area            0.003    0.001    3.149    0.002    0.245    0.245
##
## Intercepts:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##     i_gCog           -0.000    0.042   -0.000    1.000   -0.000   -0.000
##    .s_gCog            0.000    0.009    0.000    1.000    0.000    0.000
##     i_Area            0.000    0.042    0.000    1.000    0.000    0.000
##    .s_Area            0.000    0.004    0.000    1.000    0.000    0.000
##
## Variances:
##                    Estimate  Std.Err  z-value  P(>|z|)   Std.lv  Std.all
##    .gCog00            0.010                                    0.010    0.010
##    .gCog01            0.010                                    0.010    0.010
##    .gCog02            0.010                                    0.010    0.009
##    .Area0             0.010                                    0.010    0.010
##    .Area1             0.010                                    0.010    0.010
##    .Area2             0.010                                    0.010    0.010
##     i_gCog            0.975    0.059   16.495    0.000    1.000    1.000
##    .s_gCog            0.041    0.003   14.826    0.000    0.997    0.997
##     i_Area            0.982    0.059   16.539    0.000    1.000    1.000
##    .s_Area            0.003    0.000    5.826    0.000    0.954    0.954
##
## R-Square:
##                    Estimate
##     gCog00            0.990
##     gCog01            0.990
##     gCog02            0.991
##     Area0             0.990
##     Area1             0.990
```

```
##     Area2             0.990
##     s_gCog            0.003
##     s_Area            0.046

colMeans(is.na(mydata_growth[, c("Area0", "Area1", "Area2")]))

## Area0 Area1 Area2
##     0     0     0

param_est <- parameterEstimates(fit_lgm_area, standardized=TRUE)
param_edges <- param_est[param_est$op %in% c("=~", "~", "~~"), ]

param_edges$pvalue[is.na(param_edges$pvalue)] <- 1
edge_colors <- ifelse(param_edges$pvalue < 0.05, "black", "lightgrey")

edge_labels <- round(param_edges$std.all, 2)
semPaths(fit_lgm_area,
         whatLabels = "none",
         edgeLabels = as.character(edge_labels),
         edge.color = edge_colors,
         layout = "tree",
         rotation = 1,
         edge.label.cex = 0.8,
         edge.label.position = 0.6,
         sizeMan = 6,
         sizeLat = 7,
         esize = 1,
         curve = 1,
         intercepts = FALSE,
         residuals = TRUE
)
```


1
1
0.26
i_C
i_A
-0.08
0.21
1
0.95
0.24
s_C
s_A
0.99
0.98
0.93
0.99
0.99
0.98
0
0.2
0.38
0
0.05
0.11
gC00
gC01
gC02
Ar0
Ar1
Ar2
0.01
0.01
0.01
0.01
0.01
0.01